战舰世界联合作战挂机托管脚本
详细设计文档

版本：v0.4（详细设计修订） | 状态：待实机确认 | 定位：本地游戏自动化技术验证

1 文档说明 2
2 总体架构 4
3 配置设计 8
4 地图数据设计 12
5 感知模块详细设计 17
6 状态机详细设计 24
7 自动导航详细设计 30
8 防挂机行为详细设计 34
9 输入模块详细设计 37
10 退出与续场详细设计 40
11 数据与日志详细设计 43
12 标定与验证工具详细设计 47
13 异常处理与运行保障 50
14 测试与验收 52
15 开发计划 54
16 待实机确认清单 56
17 风险与运行边界 59
附录 A 核心接口签名 60
附录 B 关键流程伪代码 62

目录为 Word 域，打开文档后按 F9 或右键“更新域”刷新页码。

1 文档说明

1.1 编写目的

本文档在《战舰世界联合作战挂机脚本设计文档》v0.1 的基础上展开详细设计。初步设计确定了模块划分、状态机轮廓、地图数据组织和风险边界，本文档把这些内容细化到可以直接编码的程度，包括每个模块的职责与接口、数据结构、关键算法、配置格式、数据库表结构、标定工具、异常处理和测试方法。

本文档不包含具体代码。后续实现以本文档为依据，实现过程中发现与实机行为不符的地方，先回到本文档修订，再改代码。

1.2 已确认的设计决策

以下决策来自初步设计文档和本次确认，后续设计不再重复论证。

决策项	结论	对设计的影响
目标客户端	通用设计，优先适配国服 360，亚服 Steam 与 WG 客户端通过配置扩展	界面特征、按键、退出步骤全部放入客户端配置，代码不写死客户端专有常量
设计范围	扩展版，多地图、多出生点、多航线	数据结构和加载流程按多地图设计，实现按阶段推进
地图素材	使用外部地图图片，手工标定，底图来自 wowsdb.com 地图库	需要离线标定工具，地图识别使用特征匹配
地图方向	外部底图与游戏内小地图可能存在上下颠倒、左右镜像或 90 度旋转	地图包记录方向，识别时自动检测，标定工具提供翻转预览
配置存储	YAML 地图包为唯一权威来源	去掉 SQLite 配置表，SQLite 只记录运行日志
输入方案	ctypes 调用 SendInput 自行封装	不引入 pyautogui 与 pydirectinput
显示方式	2560×1600 无边框窗口	按窗口客户区截屏，搜索区域使用归一化坐标
截屏频率	每秒约 2 帧	主循环仍按 10 Hz 决策，感知复用最近一帧，减少截屏开销
攻击行为	开火与消耗品共用一个攻击计时器，开火占明显多数	调度器不再为消耗品单独计时，攻击事件按概率二选一
截图留存	战斗结束后删除该场次截图	截图只用于当前场次诊断，数据库不保存截图路径
船只与装备	固定一艘船，不使用信号旗和经济加成卡	启动时指定船只，港口中查找并确认，装备槽必须为空

决策项	结论	对设计的影响
运行环境	Windows 11，Python 3.14.5，OpenCV 5.0，NumPy，PyYAML	不增加重量级依赖，不引入 OCR
验证条件	游戏安装完成后进行实机验证	依赖实机画面的参数列入第 16 章待确认清单

1.3 设计范围与边界

- 包含：界面状态识别、地图与出生点识别、自动导航、防挂机随机行为、死亡判定、退出战斗、返回港口、自动续场、运行日志、离线标定工具、异常恢复。
- 不包含：开炮瞄准、锁定敌人、计算提前量、切换弹药等战斗决策；多账号管理、账号切换、代登录；游戏启动器接管；内存读取、网络封包分析、客户端文件修改。
- 客户端通用：运行时只读取客户端配置和地图包，具体界面差异通过模板、搜索区域和步骤序列表达。第一版以国服 360 客户端完成实机验证，亚服 Steam 与 WG 客户端的配置在验证后补齐。
- 分阶段实现：第一版只要求单张地图、单个出生点、单条航线跑通完整闭环；多地图、多出生点和多客户端在数据格式和架构上预留，不要求首版全部可用。
- 单机单账号单窗口：同一时间只控制一个游戏窗口，不做多开，不做账号切换。
- 固定使用一艘船只：每次会话只操作配置中的一艘船，不使用信号旗、经济加成卡或其他消耗性加成。启动时指定船只，脚本在港口中查找并确认当前选中该船。

1.4 术语与坐标约定

本文档涉及多套坐标，先统一名称和方向，后续章节不再重复说明。

术语	含义
窗口客户区	游戏画面在桌面上的矩形区域，不含标题栏和边框。所有截屏和点击都以这个区域为基准
世界坐标	地图逻辑坐标，单位米，x 轴向东，y 轴向北。出生点、航点、舰船位置都使用这套坐标
底图像素坐标	外部地图图片的像素坐标，原点在左上角，y 轴向下
小地图像素坐标	游戏小地图内容区域的像素坐标，原点在小地图内容区左上角
屏幕坐标	整个桌面的物理像素坐标，用于鼠标绝对点击
归一化坐标	相对窗口客户区的比例坐标，取值 0～1，用于描述搜索区域和小地图位置，避免分辨率变化导致配置失效
特征	界面上的一个可识别元素，由模板图片、搜索区域、匹配阈值和缩放范围组成

术语	含义
地图包	一张地图的底图、元数据、出生点和航线的集合，存放在一个目录中
客户端配置	一个客户端版本的窗口识别方式、界面特征、按键映射和操作步骤
随机档案	每场战斗开始时生成的一组行为参数快照，用于让每场战斗的行为有差异

1.5 证据缺口与文档使用方式

撰写本文档时游戏尚未完成安装，无法获取真实界面截图。本文档中所有界面特征的位置、模板内容、匹配阈值、死亡判定信号、退出步骤和按键，都是基于公开资料的初始假设，必须通过实机截图和操作确认后才能定值。

为了区分确定内容和待验证内容，本文档用“待验证”标记尚未实机确认的参数，并在第 16 章汇总成清单，每项都给出验证方法。实现阶段先按初始假设编码，同时把参数做成配置项，实机确认后只改配置，不改代码。

小地图是否可点击、是否北向固定、死亡后能否立即退出、退出入口在哪个菜单，这些属于游戏现有行为，公开资料存在版本差异，本文档只给出设计假设，不作为事实陈述。

2 总体架构

2.1 运行形态

程序是一个普通 Windows 桌面进程，使用 Python 3.14 运行，控制同一台机器上的一个游戏窗口。游戏以 2560×1600 无边框窗口运行，脚本运行期间游戏窗口保持在前台。程序不注入游戏进程，不读取游戏内存，不修改客户端文件，不抓取网络数据，只通过截屏和模拟键鼠工作。

启动流程：加载全局配置和客户端配置，加载并校验地图包，查找游戏窗口，检查截屏和输入是否可用，进入主循环。主循环先识别当前界面，再进入对应状态。停止时释放所有按下的键和鼠标按钮，写完日志和数据库记录，关闭进程。

运行入口是命令行程序，例如 `python -m src.main --profile cn360`。程序不提供主界面，运行状态通过控制台和日志查看，操作通过紧急停止和恢复热键完成。标定工具单独运行，不受主程序影响。

运行期间需要操作系统保持登录状态，屏幕不锁定、不休眠，游戏窗口不被其他窗口遮挡。这些是运行前提，程序无法在锁屏或休眠状态下继续工作。

2.2 模块划分

模块	职责	主要接口	依赖
感知模块	截屏、模板匹配、地图识别、舰船位置识别、死亡信号检测	PerceptionEngine.capture / match_feature / identify_map / locate_own_ship / check_death	OpenCV、NumPy、Win32 截屏
状态机	维护当前状态，按事件和超时驱动转移，管理战斗内子流程	StateMachine.step(observation, now) → 动作列表	感知、执行、存储
执行模块	把动作翻译成键鼠事件，串行执行，处理窗口焦点	InputExecutor.submit(action)	SendInput
导航模块	选择航线、计算点击坐标、确认舰船移动、处理卡住	Navigator.tick(observation, now) → 动作列表	感知、执行、地图数据
行为模块	按随机调度生成视角和攻击动作，攻击动作在开火与消耗品之间选择	BehaviorScheduler.tick(now) → 动作列表	执行、随机档案
存储模块	加载 YAML 配置，校验地图包，写 SQLite 日志	ConfigLoader / Database	PyYAML、sqlite3
工具模块	地图标定、界面模板截取、画面录制、离线回放、环境检查	命令行入口	OpenCV、tkinter

2.3 模块依赖与数据流

感知模块和存储模块位于最底层，不依赖任何业务模块。导航模块和行为模块依赖感知模块与执行模块。状态机依赖感知、导航、行为和执行，负责决定当前应该运行哪些子流程。工具模块独立于运行时，可以单独运行。

数据流是单向的：截屏产生图像画面，感知模块把画面转换成结构化观察结果，状态机根据观察结果和当前状态决定动作，动作交给执行模块产生键鼠事件，动作和状态变化同时写入日志模块。导航和行为不直接调用键鼠接口，也不直接写数据库。

禁止反向依赖：感知模块不读取状态机状态，执行模块不读取配置中的业务规则，工具模块不被运行时导入。这样每个模块都可以单独测试和替换。

2.4 线程与调度模型

程序使用四个线程，职责固定，共享数据尽量少。

线程	职责	运行频率	共享数据
主线程	截屏调度、感知、状态机、导航和行为决策	决策 10 Hz；截屏固定 2 Hz	当前观察结果、状态机状态、子流程对象

线程	职责	运行频率	共享数据
输入线程	消费动作队列，按顺序执行键鼠事件，处理按下与释放的配对	由动作触发，最小点击间隔 150 ms	线程安全动作队列、紧急停止标志
日志线程	消费事件队列，批量写入 SQLite，写失败时降级到文本日志	队列有数据时立即写，最多 1 秒一批	线程安全事件队列、数据库连接
看门狗线程	检查主循环心跳、紧急停止热键、会话时长上限	20 Hz	心跳时间戳、停止标志

主线程负责所有决策，避免状态分散在多个线程中。输入线程是唯一的键鼠出口，导航点击、视角移动、开火和消耗品按键都排进同一个队列，保证不会出现鼠标按下和移动交叉、点击被视角移动打断的情况。日志线程独立，避免数据库写入抖动影响状态机节奏。

超时和行为间隔使用单调时钟计算，不使用连续 sleep 累加，避免长时间运行后时间漂移。主线程每轮记录心跳时间，看门狗检测到超过 10 秒没有心跳时，停止输入并进入暂停状态。

2.5 项目目录结构

```
wows_script/
├── config.yaml                    # 全局运行配置
├── profiles/                      # 客户端配置
│   ├── cn360/
│   │   ├── profile.yaml           # 窗口、特征、退出步骤
│   │   ├── keys.yaml              # 按键映射
│   │   ├── features/              # 界面模板图片
│   │   └── ships/
│   │       └── <ship_id>/
│   │           ├── ship.yaml      # 船只信息、卡片模板和装备校验配置
│   │           ├── card.png       # 港口船只卡片模板
│   │           └── loadout/       # 空槽校验模板，可选
│   ├── steam/
│   └── wgc/
├── maps/                          # 地图包
│   └── <map_id>/
│       ├── background.png         # 外部地图底图
│       ├── map.yaml               # 地图元数据与坐标标定
│       ├── spawns.yaml            # 出生点
│       ├── routes.yaml            # 航线
│       └── features.npz           # 可选，预计算的特征缓存
├── data/
│   └── wows_script.db             # SQLite 运行日志
├── logs/                          # 文本日志
├── screenshots/                   # 异常与关键节点截图
├── src/
│   ├── main.py                    # 程序入口
│   ├── config/                    # 配置加载与校验
│   ├── perception/                # 截屏与识别
│   ├── state_machine/             # 状态机与子流程
```

```
│   ├── execution/          # SendInput 封装与输入队列
│   ├── navigation/         # 导航
│   ├── behavior/           # 随机行为调度
│   ├── storage/            # SQLite 与日志
│   ├── tools/              # 标定、录制、回放、检查
│   └── common/             # 数据类型、坐标变换、工具函数
└── tests/
```

2.6 技术选型

需求	选择	理由	放弃的方案与原因
截屏	ctypes 调用 GDI BitBlt，持久化内存 DC	不增加依赖，可以只取窗口客户区或指定区域，2560×1600 下单帧约 10～20 ms	Desktop Duplication 实现复杂；mss 需要额外依赖，窗口裁剪和坐标换算仍要自己做
图像处理	OpenCV 与 NumPy	本机已安装，模板匹配、特征匹配、坐标变换齐全，性能足够	不自行实现图像算法
界面识别	模板匹配为主，按状态查询指定特征	界面文字和按钮样式固定，模板匹配速度快，误判可通过阈值和连续帧确认控制	不引入 OCR：中文 OCR 需要额外模型和依赖，字体渲染差异会带来识别率问题
地图识别	ORB 特征匹配加局部仿射变换	外部底图与游戏小地图比例不同，特征匹配可以同时确定地图身份和坐标变换	纯模板匹配对外部底图不成立；颜色直方图区分度不足
配置	YAML	可手工编辑，便于标定、复查和版本管理	SQLite 不适合人工维护和版本对比
日志	SQLite 加文本日志	结构化数据便于按场次查询，文本日志便于快速查看	只用文本日志难以做统计和关联查询
标定界面	tkinter	Python 自带，离线工具对界面要求不高	Qt 增加体积和依赖
输入	ctypes 调用 SendInput	原生接口，支持绝对点击和相对移动，事件可控	pyautogui 以绝对移动为主，pydirectinput 增加依赖且维护有限

Python 3.14 与 OpenCV 5.0 的组合需要在实现阶段先做一次兼容性检查，重点是 ctypes 结构体定义、NumPy 数组与 OpenCV 图像的转换、tkinter 的显示。检查不通过时，优先调整依赖版本，不改变架构。

3 配置设计

3.1 配置分层

配置分为四层，职责不同，加载顺序固定。

- 全局配置 `config.yaml`：运行参数、路径、阈值、超时、安全限制和会话限制。
- 客户端配置 `profiles/<client_id>/profile.yaml`：窗口识别、界面特征、退出步骤、开局步骤和语言。
- 按键配置 `profiles/<client_id>/keys.yaml`：动作到按键的映射，特别是消耗品栏位。
- 地图包 `maps/<map_id>/`：地图元数据、出生点、航线。

加载顺序为全局配置、客户端配置、地图包。后加载的内容覆盖前者的同名默认值，但不允许客户端配置覆盖路径和安全限制，避免误配置导致脚本去操作别的窗口。所有相对路径以项目根目录为基准解析。

运行前必须指定一个客户端配置和一个地图包集合。地图包可以一次加载多个，运行时根据识别结果选择；也可以通过配置强制指定地图，用于标定和调试。

3.2 客户端配置

客户端配置描述“这个客户端长什么样、怎么操作”，不包含任何战局逻辑。更换客户端时只替换配置和模板图片。

字段	类型	说明
client_id	字符串	客户端标识，例如 cn360、steam、wgc
display_name	字符串	显示名称，写入日志
window.process_names	字符串列表	游戏进程名，用于查找窗口
window.title_patterns	字符串列表	窗口标题正则，进程名找不到时使用
window.require_foreground	布尔值	是否要求游戏窗口在前台，默认 true
language	字符串	界面语言，zh_cn 或 en，用于选择模板组
reference_resolution	两个整数	模板采集时的窗口尺寸，默认 2560×1600
features	映射	界面特征定义，见下方示例
start_sequence	步骤列表	从港口进入联合作战并开始匹配的步骤
exit_sequence	步骤列表	死亡后退出到港口的步骤
popup_sequence	步骤列表	退出后清理弹窗的步骤

字段	类型	说明
ship_selection_sequence	步骤列表	在港口打开船只列表、滚动查找并选择配置船只的步骤
ship_verify_features	特征列表	确认当前选中的是配置船只
loadout_verify_features	特征列表	确认信号旗和加成卡槽位为空
clear_loadout_sequence	步骤列表或 null	可选的装备清除步骤，默认 null，不自动清除
keys_file	字符串	按键配置文件路径，默认同目录 keys.yaml

界面特征的定义包含模板图片、搜索区域、阈值和缩放范围。搜索区域使用归一化坐标，窗口尺寸变化时不需要修改配置。

```
features:
  port_battle_button:
    template: features/port_battle_button.png
    region: [0.55, 0.82, 0.98, 0.99]   # x1, y1, x2, y2，相对窗口客户区
    threshold: 0.85
    scales: [0.95, 1.0, 1.05]

  death_banner:
    template: features/death_banner.png
    region: [0.25, 0.30, 0.75, 0.70]
    threshold: 0.80
    scales: [0.95, 1.0, 1.05]
```

开局、退出和弹窗处理使用同一种步骤序列，步骤类型包括按键、点击特征、等待特征出现、等待固定时间。每个步骤可以设置超时和是否可选。

```
exit_sequence:
  - action: key
    key: escape
    wait_after_ms: 500
  - action: click_feature
    feature: exit_menu_return_port
    timeout_ms: 3000
  - action: click_feature
    feature: exit_confirm_yes
    optional: true
    timeout_ms: 2000
  - action: wait_feature
    feature: port_indicator
    timeout_ms: 120000
```

步骤序列由通用执行器解释，不在代码中为某个客户端写专用分支。客户端如果退出路径不同，只改配置。步骤失败时先重试当前步骤，连续失败则进入异常恢复。

3.3 全局配置

全局配置按用途分组，下面列出主要字段。默认值是在 2560×1600、无边框窗口、国服客户端条件下给出的初始建议，实机验证后调整。

配置组	字段	默认值	说明
runtime	loop_hz	10	主循环决策频率
runtime	capture_hz	2	截屏频率，每秒约 2 帧
runtime	debug	false	是否输出调试日志和中间图像
paths	data_dir / log_dir / screenshot_dir	data / logs / screenshots	运行产物目录
map	auto_identify	true	是否自动识别地图
map	forced_map_id	null	强制指定地图，用于调试
map	spawn_tolerance_m	800	出生点匹配的最大距离
navigation	surface	minimap	导航点击面，minimap 或 full_map
navigation	arrival_m	400	判定到达航点的距离
navigation	click_retry	2	点击后未移动的重试次数
navigation	stuck_window_s	15	卡住判定的观察窗口
navigation	stuck_distance_m	80	观察窗口内允许的最小位移
behavior	camera_interval_s	[3, 15]	视角动作间隔范围
behavior	attack_interval_s	[25, 120]	攻击事件间隔范围，开火和消耗品共用
behavior	consumable_ratio	0.15	每次攻击事件选择消耗品的概率，其余选择开火
behavior	consumable_max_per_battle	10	单场消耗品点击上限
behavior	activity_watchdog_s	[35, 95]	长时间无操作后的兜底动作间隔
ship	ship_id	必填	会话固定使用的船只编号，对应 ships/<ship_id>/
ship	verify_each_battle	true	每场开始前是否重新确认船只和空装备
safety	emergency_stop_key	f12	紧急停止热键
safety	max_session_minutes	240	单次运行最长时间
safety	max_battles_per_session	20	单次运行最多场次
safety	pause_when_unfocused	true	窗口失去焦点时是否暂停

配置组	字段	默认值	说明
session	between_battle_s	[15, 60]	两场之间的随机等待
session	daily_battle_cap	30	每日场次上限，跨进程累计
session	allowed_hours	null	允许运行的时段，例如 08:00-23:00，null 表示不限
timeouts	queue_s / loading_s	600 / 300	排队和加载超时
timeouts	exit_menu_s / exit_confirm_s	5 / 5	退出菜单和确认框超时
timeouts	exit_s / port_s	180 / 180	退出和回港超时
timeouts	popup_s / recovery_s	60 / 30	弹窗清理和异常恢复超时
timeouts	heartbeat_s	10	主循环心跳超时
logging	screenshot_on_error	true	异常时保存截图，仅用于当前场次
logging	delete_screenshots_after_battle	true	战斗结束后删除该场次截图目录

3.4 按键映射

按键映射单独成文件，原因是消耗品键位在客户端之间和玩家之间都可能不同。配置里保存实际按键，不假设默认键位。

```
actions:
  escape: escape
  fire: mouse_left
  full_map: m
  camera_relative: true          # 视角使用鼠标相对移动

consumables:
  slots:
    - key: "1"
      name: "损管小组"
    - key: "2"
      name: "维修小组"
    - key: "3"
      name: "水听"
    - key: "4"
      name: "引擎增压"
```

启动时校验按键名称是否在支持列表中，消耗品列表是否为空。按键名称对应的虚拟键码和扫描码在输入模块中维护，配置只出现可读名称。

消耗品采用纯随机点击，不做血量或状态判断，因此键位配置必须和游戏内设置一致。第一次运行时通过实机测试确认每个栏位按下的实际效果。

3.5 配置加载与校验

配置使用 PyYAML 的 safe_load 读取。加载后执行三类校验：结构校验检查字段是否存在、类型是否正确；范围校验检查频率、距离、超时、概率是否在允许区间；引用校验检查模板文件是否存在、图片能否读取、出生点和航线的编号是否互相匹配。

校验失败时一次性列出所有问题，包含文件路径和字段路径，程序以退出码 2 结束，不进入主循环。这样可以在开机前发现配置错误，避免脚本运行到一半才失败。

提供 `python -m src.tools.validate` 命令，只做校验不启动脚本，便于修改配置后快速检查。配置文件和地图包修改后，建议先运行一次校验。

3.6 配置覆盖规则

客户端配置可以覆盖全局配置中的界面相关默认值，例如搜索区域和超时；不能覆盖路径、安全限制和会话上限，这些字段只从全局配置读取。地图包只提供地图数据，不参与运行参数。

覆盖关系在加载时记录下来，写入日志。出现问题时可以从日志看到每个参数最终来自哪个文件，避免排查时猜测。

4 地图数据设计

4.1 地图包结构

地图包是脚本运行所需地图数据的唯一来源。每张地图一个目录，目录名就是地图编号，包含底图、元数据、出生点和航线。外部底图用于航线规划、坐标标定，并作为地图识别的特征来源，不作为像素级模板直接匹配。

```
maps/<map_id>/
├── background.png      # 外部地图底图，建议保留原始分辨率
├── map.yaml            # 地图元数据与图像坐标标定
├── spawns.yaml         # 出生点列表
├── routes.yaml         # 航线列表
└── features.npz        # 可选，预计算的 ORB 特征，缺失时启动阶段生成
```

地图编号使用英文小写和下划线，避免空格和中文，例如 `north_sea`、`islands_of_ice`。编号一旦使用不随显示名称变化，日志和数据库都以编号记录。

外部底图统一从 wowsdb.com 地图库下载，地址为 `https://wowsdb.com/zh-CN/database/maps`。下载后按地图编号建立目录，保存为 `background.png`，再按 4.2 节标定

可游玩区域。地图库图片的分辨率、边框、图例和使用条款需要在 M0 阶段核对；带有边框或图例的图片必须在标定时排除，否则像素坐标和世界坐标的对应会整体偏移。

4.2 地图元数据 map.yaml

map.yaml 描述底图和世界坐标的对应关系。外部图片通常带有图例、边框或空白边距，因此需要单独标出可游玩区域在图片中的像素矩形。

字段	类型	说明
map_id	字符串	地图编号，必须与目录名一致
name	字符串	地图显示名称，仅用于日志和标定界面
size_meters	两个数字	地图可游玩区域的宽度和高度，单位米
world_bounds	四个数字	世界坐标范围 x_min, y_min, x_max, y_max，单位米
image_bounds	四个数字	可游玩区域在底图中的像素范围 u_min, v_min, u_max, v_max
image_orientation	字符串	底图相对世界坐标的方向，取值 normal、rot90、rot180、rot270、mirror_h、mirror_v、mirror_rot90、mirror_rot270
world_to_image	六个数字或 null	可选的直接仿射矩阵；提供时优先于 bounds 和 orientation
default_minimap_rect	四个数字或 null	该地图推荐的小地图归一化区域，可留空使用客户端配置
notes	字符串	标定备注，例如底图来源和版本

```
map_id: north_sea
name: 北方海域
size_meters: [12000, 12000]
world_bounds: [-6000, -6000, 6000, 6000]
image_bounds: [120, 80, 1880, 1840]
image_orientation: normal
world_to_image: null
default_minimap_rect: null
notes: 底图来自地图站，裁掉了四周图例
```

world_bounds 的取值取决于底图采用的坐标原点。使用中心原点或左上角原点都可以，后续所有出生点和航点必须使用同一套范围。标定工具会显示当前范围，避免混用。

size_meters 与 world_bounds 的宽高应当一致。校验时如果两者差值超过 1%，给出警告，提示底图比例可能标错。

4.3 出生点 spawns.yaml

出生点使用世界坐标，并记录初始朝向。朝向以正北为 0 度，顺时针增加，单位为度。朝向用于选择航线和估算初始视角，不参与点击坐标计算。

```yaml
spawns:
  - spawn_id: east_1
    position: [4200, -3500]
    facing_deg: 270
    label: 东侧一号位

  - spawn_id: west_1
    position: [-4300, 3200]
    facing_deg: 90
    label: 西侧一号位
```

出生点编号在同一张地图内唯一。校验规则：坐标必须落在 world_bounds 内；两个出生点之间的距离小于 500 米时给出警告，因为识别时可能混淆。

4.4 航线 routes.yaml

航线绑定出生点，由若干航点组成。航点按顺序点击，最后一个航点到达后按配置决定后续行为。同一条出生点可以配置多条航线，运行时随机选择。

```yaml
routes:
  - route_id: east_1_south
    spawn_id: east_1
    weight: 1.0
    description: 沿东侧岛屿向南航行
    waypoints:
      - [3000, -2800]
      - [1800, -1200]
      - [1200, 600]
      - [600, 2500]

  - route_id: east_1_center
    spawn_id: east_1
    weight: 0.6
    description: 向地图中心航行
    waypoints:
      - [2500, -2000]
      - [1000, -500]
      - [0, 1000]
```

字段	类型	说明
route_id	字符串	航线编号，全地图唯一
spawn_id	字符串	绑定的出生点编号
weight	数字	同出生点多条航线时的选择权重，默认 1.0
description	字符串	航线说明，便于人工检查
waypoints	坐标列表	世界坐标航点，按顺序排列

航线规则：第一个航点与出生点的距离不超过 1500 米；相邻航点距离在 200 米到 10000 米之间；所有航点位于 world_bounds 内；每条航线至少两个航点。

航点是否落在水域内由标定人员负责，程序不做地形判断。底图上绘制航线时应当避开岛屿，宁可绕行也不要贴着岸边，因为小地图点击的精度有限，贴岸容易搁浅。

航线长度建议控制在 3～10 个航点。航点过多会增加点击次数和出错机会，过少会让航行路线过于机械。

4.5 坐标系统与变换

地图数据涉及四层坐标，变换顺序是固定的。

- 世界坐标：米为单位的逻辑坐标，出生点、航点、舰船位置使用这一层。
- 底图像素坐标：外部底图的像素坐标，原点在左上角。
- 小地图像素坐标：游戏小地图内容区域的像素坐标。
- 屏幕坐标：桌面的物理像素坐标，鼠标点击使用这一层。

世界坐标到底图像素坐标的变换由 world_bounds 和 image_bounds 决定。底图上方为北方时，x 向东增大对应像素 u 向右增大，y 向北增大对应像素 v 向上减小。

```
u = u_min + (x - x_min) / (x_max - x_min) * (u_max - u_min)
v = v_min + (y_max - y) / (y_max - y_min) * (v_max - v_min)
T = orientation_matrix * bounds_scale * translate
[u, v, 1]^T = T * [x, y, 1]^T
```

外部底图可能相对游戏内小地图上下颠倒、左右镜像或旋转 90 度的整数倍。地图包用 image_orientation 记录底图方向，标定工具和运行时识别都使用这个方向。世界坐标到底图像素坐标的变换统一写成一个 2×3 仿射矩阵，由 world_bounds、image_bounds 和 image_orientation 计算；提供了 world_to_image 时直接使用该矩阵。

方向变化不改变世界坐标本身。出生点和航点始终用世界坐标保存，翻转和旋转只影响它们显示在底图上的像素位置，以及世界坐标到小地图的变换矩阵。标定工具预览时必须先确认方向，再放置出生点和航点。

运行时地图识别会重新估计方向。识别结果与 map.yaml 声明不一致时，以识别结果为准并记录警告；如果识别出的旋转角不是 90 度的整数倍，说明小地图可能随镜头旋转，暂停导航并要求在游戏设置中固定北向。

世界坐标到小地图像素坐标有两种模式。第一种是整图模式：小地图显示整张地图，且北向固定，直接按地图边界线性映射。第二种是匹配模式：用地图识别得到的仿射变换把底图像素坐标映射到小地图坐标。两种模式最终都得到一个从世界坐标到小地图像素的仿射矩阵。

整图模式：

```
mx = m_min_x + (x - x_min) / (x_max - x_min) * (m_max_x - m_min_x)
my = m_min_y + (y_max - y) / (y_max - y_min) * (m_max_y - m_min_y)
```

匹配模式：

```
[u, v] = world_to_image(x, y)
[mx, my] = A * [u, v, 1]      # A 由特征匹配估计
```

小地图内容区域用归一化坐标描述，例如 `[0.86, 0.03, 0.99, 0.24]`，换算成窗口客户区像素后再加窗口左上角的屏幕坐标，得到最终点击位置。归一化坐标可以适配窗口尺寸变化。

小地图是否显示整张地图、是否北向固定，需要实机确认。设计上第一版要求整图且北向固定，如果游戏设置可以旋转或缩放小地图，必须先在游戏里改为固定模式，否则导航坐标不可靠。

4.6 航线选择与会话内变化

战斗中识别出地图和出生点后，从绑定该出生点的航线中按权重随机选择一条。随机数使用本场战斗的随机种子，同一种子可以复现选择结果，便于复盘。

如果出生点没有配置航线，先检查该地图是否有默认航线；都没有时进入异常恢复，不使用其他出生点的航线强行替代。把船开到不熟悉的路线会显著增加搁浅和卡住的概率。

航线最后一点到达后有三种行为，由全局配置 `navigation.route_end_action` 决定：原地等待（默认）、按原路返回、在当前点附近随机选一个新的等待点。原地等待最简单，脚本停在安全水域，等待被击沉或战斗结束。

同一出生点配置多条航线，可以让不同场次的航迹不完全一致。权重可用于控制某条航线出现的比例，调试阶段可以把不成熟的航线权重设为 0。

4.7 地图数据校验

地图包在启动时和标定工具保存时都会校验。校验失败阻止运行，校验警告写入日志。

检查项	级别	处理方式
目录名与 map_id 一致	错误	拒绝加载
底图存在且可读取	错误	拒绝加载
world_bounds 宽高与 size_meters 一致	警告	继续运行，日志提示
image_bounds 在底图范围内	错误	拒绝加载

检查项	级别	处理方式
出生点在 world_bounds 内	错误	拒绝加载
出生点间距小于 500 米	警告	日志提示，可能识别混淆
航线引用的出生点存在	错误	拒绝加载
航点数量少于 2	错误	拒绝加载
航点超出 world_bounds	错误	拒绝加载
首航点距出生点超过 1500 米	警告	日志提示
相邻航点距离超过 10000 米	警告	点击精度可能不足
航线编号重复或出生点编号重复	错误	拒绝加载
image_orientation 取值合法	错误	拒绝加载
world_to_image 不可逆或与 bounds 冲突	警告	以 bounds 和 orientation 重新计算

5 感知模块详细设计

5.1 模块职责与接口

感知模块只回答“画面上有什么”，不做决策。它接收截屏请求，返回模板匹配结果、地图识别结果、舰船位置和死亡信号。所有结果带时间戳和置信度，状态机根据置信度决定是否采信。

```
@dataclass(frozen=True)
class MatchResult:
    feature_id: str
    found: bool
    score: float
    center: tuple[int, int] | None
    rect: tuple[int, int, int, int] | None
    scale: float
    timestamp: float

@dataclass(frozen=True)
class MapMatch:
    map_id: str
    inliers: int
    score: float
    affine: tuple[float, ...]          # 底图像素到小地图像素的 2x3 仿射矩阵
    reprojection_error: float
    timestamp: float

@dataclass(frozen=True)
class ShipPosition:
    minimap_px: tuple[float, float]
    world_xy: tuple[float, float]
    heading_deg: float | None
    speed_mps: float
```

```
    confidence: float
    source: str                         # marker / predicted / lost

@dataclass(frozen=True)
class DeathSignals:
    hp_bar_absent: bool
    hp_zero: bool
    banner: bool
    marker_lost: bool
    spectator_ui: bool
    score: float
    stable_ms: int
    timestamp: float

@dataclass(frozen=True)
class Observation:
    timestamp: float
    window_rect: tuple[int, int, int, int]
    features: dict[str, MatchResult]
    state_guess: str | None
    map_match: MapMatch | None
    ship: ShipPosition | None
    death: DeathSignals | None
```

PerceptionEngine 对外提供五个方法：`capture(region)` 截取画面，`match_feature(feature_id)` 匹配单个特征，`detect_state()` 在恢复流程中做整体判断，`identify_map()` 识别地图并估计仿射变换，`locate_own_ship()` 定位自身舰船。死亡信号由 `check_death()` 汇总，单独返回。

5.2 屏幕捕获

捕获使用 Windows 的 GDI 接口。先用进程名和窗口标题找到游戏窗口，取得窗口客户区矩形，再通过窗口 DC 把画面复制到内存位图，最后转换成 NumPy 数组。整个过程不注入游戏进程。

```
查找窗口：
  EnumWindows → GetWindowThreadProcessId → QueryFullProcessImageNameW
  匹配 window.process_names；找不到时按 window.title_patterns 匹配标题

取图：
  GetClientRect(hwnd)           # 客户区尺寸
  ClientToScreen(hwnd)          # 客户区左上角的屏幕坐标
  GetDC(hwnd) → CreateCompatibleDC → CreateCompatibleBitmap
  BitBlt(SRCCOPY | CAPTUREBLT)
  GetDIBits → BGRA 缓冲区 → NumPy 数组 → 丢弃 alpha
```

程序启动时声明 DPI 感知，避免系统缩放把截屏尺寸和点击坐标搞混。内存 DC 和位图在窗口尺寸不变时复用，尺寸变化时重新创建。

捕获前检查窗口存在、未最小化、客户区尺寸大于零。游戏窗口被其他窗口遮挡时，BitBlt 可能取到遮挡窗口的内容，因此运行期间要求游戏窗口在前台，失焦时暂停而不是继续识别。

截屏频率固定为每秒约 2 帧，由 `runtime.capture_hz` 配置，默认值为 2。主循环仍按 10 Hz 运行，状态机、导航和行为计时每轮执行，感知结果在没有新帧时复用最近一次结果。每次截取完整窗口客户区，小地图、血条和提示区域都从这一帧裁剪，不重复截屏。每次捕获记录耗时，超过 300 毫秒时写警告并跳过该帧，避免卡顿累积。

截屏失败或窗口消失时抛出 `CaptureUnavailable`，状态机收到后进入异常恢复，不把失败画面当作有效观察。

5.3 图像预处理

模板匹配在灰度图上进行。模板在加载时转换为灰度，并预先生成 0.95、1.0、1.05 三个缩放版本，避免每帧重复缩放。匹配使用归一化相关系数，光照均匀变化时仍然稳定。

舰船标记检测需要颜色信息，使用 HSV 阈值和模板两种方式结合。HSV 阈值在客户端配置中给出范围，模板由标定工具从实机截图中截取。颜色阈值只作为候选筛选，最终位置以模板得分和时序跟踪为准。

所有模板在启动时校验：文件存在、尺寸不小于 8×8 像素、不是纯色图。纯色模板在归一化匹配中会产生大量误匹配，校验时拒绝加载。

5.4 界面特征识别

界面识别使用模板匹配，按状态查询需要的特征，不做全屏分类。例如在港口状态只查询联合作战入口和开始战斗按钮，在战斗中只查询 HUD 标志、血条和死亡提示。这样匹配区域小、速度快，也不容易被其他界面的相似元素干扰。

初步设计把 OCR 列为感知模块能力，详细设计改为模板匹配。界面上的按钮、提示和标题在单个客户端内是固定渲染，模板匹配的稳定性高于中文 OCR，也不需要额外模型和依赖。后续如果出现必须读取动态文字的场景，例如地图名称没有固定模板，再单独增加 OCR 模块，接口保持为“输入区域、输出文字”，不影响现有特征匹配。

匹配流程：把归一化搜索区域换算成窗口像素区域，裁剪图像，在多个缩放比例下执行 `matchTemplate`，取最高分。分数超过阈值才算命中。状态切换要求连续两帧命中，避免动画和渐变造成抖动。

状态	主判定特征	辅助特征	说明
PORT	port_indicator	port_battle_button、port_mode_selector、coop_mode_selected	港口主界面可见，并确认当前战斗模式为联合作战
QUEUEING	queue_indicator	cancel_queue_button	匹配队列界面
LOADING	loading_indicator	loading_tip_area	加载画面
BATTLE_ACTIVE	battle_hud_indicator	hp_bar_present、minimap_frame	战斗 HUD 与小地图可见
DEAD	death_banner	hp_bar_absent、spectator_indicator	死亡相关信号，见 5.9
BATTLE_FINISHED	battle_results	result_panel	战斗结束结算界面
EXIT_MENU_OPEN	exit_menu_panel	exit_menu_return_port	ESC 菜单已打开
EXIT_CONFIRMING	exit_confirm_dialog	exit_confirm_yes	退出确认框
POST_EXIT_INTERFACE	popup_close_button	reward_panel、mission_panel	回港后的弹窗

`detect_state()` 只在异常恢复时使用。它按优先级依次检查弹窗、退出菜单、死亡、战斗、加载、排队、港口，第一个命中的特征作为当前状态。优先级保证模态窗口不会被下层界面掩盖。

连续两秒无法识别出任何已知状态时，状态机进入异常恢复。感知模块不猜测状态，返回 unknown，由状态机决定下一步。

5.5 地图识别

地图识别同时解决两个问题：当前是哪张地图，以及底图像素和小地图像素之间的几何关系。使用 ORB 特征匹配加局部仿射变换，比纯模板匹配更适合外部底图。

启动时对每张地图的底图提取 ORB 特征，把关键点和描述子缓存到 `features.npz`。缓存记录底图文件的修改时间和大小，底图变化后自动重建。底图特征数量默认 2000，保留足够的地形细节。

底图缓存同时保存原图和水平镜像图的特征。旋转不需要额外缓存，ORB 描述子本身对旋转不敏感；镜像无法用旋转表达，所以单独保存一份镜像特征。

```
运行时识别：
1. 裁剪小地图内容区域，转灰度
2. 提取 ORB 特征，数量 1000
3. BFMatcher(HAMMING) + KNN(k=2)，Lowe 比例筛选 0.75
4. estimateAffinePartial2D(RANSAC, 重投影阈值 3.0)
5. 统计内点数、缩放、旋转角、重投影误差
```

6. 根据仿射矩阵计算旋转角，判断是否接近 90 度的整数倍，并记录是否使用了镜像特征

接受条件：内点数不少于 15；缩放比例与预期小地图比例相差不超过 50%；旋转角接近 0、90、180 或 270 度，允许 ±10 度误差；重投影误差中位数不超过 2.5 像素。多个地图同时满足时，选择内点数最多且与第二名有明显差距的一个，差距不足 30% 时记录警告。

底图可能左右镜像。启动时同时提取原图和水平镜像图的 ORB 特征，运行时分别匹配，选择内点更多的一方。镜像匹配得到的仿射矩阵先经过镜像坐标变换，再与世界坐标到图像坐标的矩阵合成。

识别结果同时给出旋转角是否接近 90 度的整数倍、是否发生镜像。与 map.yaml 的 image_orientation 不一致时，以识别结果为准并记录警告；连续多场不一致时提示重新标定。旋转角明显不是 90 度的整数倍时，视为小地图随镜头旋转，暂停导航。

地图识别失败后的回退顺序：连续三帧重试；使用加载界面上的地图名称特征识别；使用配置中强制指定的地图；进入异常恢复。回退过程中记录当前小地图截图，便于离线分析。

识别成功后得到的仿射矩阵只在当前战斗有效。窗口尺寸变化、小地图设置变化或识别置信度下降时重新计算。地图识别不需要每帧执行，战斗开始时执行一次，之后每 30 秒或导航异常时刷新。

5.6 小地图标定与运行时校验

小地图的位置和大小来自客户端配置的归一化区域，边框宽度单独配置。运行时根据当前窗口客户区换算成像素矩形，再裁掉边框得到内容区域。

如果配置只给出大致位置，启动时可以用小地图边框模板做一次精确定位：在配置区域附近搜索 minimap_frame 模板，用匹配结果修正矩形。边框不明显或模板匹配失败时，使用配置值并记录警告。

标定分为两种模式。整图模式假设小地图显示整张地图且北向固定，直接按地图边界线性映射。匹配模式使用地图识别的仿射矩阵，把世界坐标经过底图像素坐标映射到小地图。匹配模式的精度依赖特征匹配质量，整图模式依赖小地图确实显示全图。

运行时校验：识别出的出生点位置与配置出生点的距离应小于容差；导航点击后舰船位置变化方向应与目标方向一致。任一校验失败时重新识别地图，连续失败则暂停导航并记录警告，不继续盲目点击。

如果实机确认小地图会随镜头旋转，第一版不实现旋转补偿，直接要求用户在游戏设置中关闭小地图旋转。旋转补偿需要实时得到镜头朝向，现有感知能力不足，强行实现会引入更大的误差。底图相对小地图的静态翻转或旋转不属于这种情况，按 4.5 节和 5.5 节处理。

5.7 自身舰船位置识别

自身舰船在小地图上有一个固定样式的标记。标定工具从实机截图中截取该标记，保存为模板。运行时在小地图内容区域内做多角度模板匹配，角度从 0 到 345 度，每 15 度一个版本，取最高分作为候选位置和朝向。

模板匹配之外增加两条约束。第一条是颜色约束：标记颜色范围在客户端配置中给出，候选位置周围的颜色比例不达标时丢弃。第二条是时序约束：根据上一帧位置和估计速度预测当前位置，优先选择距离预测位置较近的候选，避免把友军或敌军的相似标记误认为自身。

小地图上友军、敌军和自身标记的形状可能相似，仅靠模板容易混淆。时序跟踪是主要的区分手段，因为只有自身标记会按照自己的速度连续移动。连续三帧位置跳变超过 300 像素时，清空跟踪状态重新搜索。

识别结果通过当前标定的逆变换转换为世界坐标。速度由最近 3～5 秒的位移中位数估算，不使用游戏显示的航速数值，避免再增加一个文字识别环节。

连续 3 秒无法定位时标记为 lost，导航进入降级模式：用最后已知位置和速度外推，同时继续尝试重新定位。连续 10 秒无法定位时停止导航，记录警告并等待死亡或战斗结束，不再发送点击。

5.8 出生点识别

战斗开始后等待小地图和 HUD 稳定，再定位自身舰船。得到世界坐标后，计算它到该地图所有出生点的距离，取最近的一个。距离小于配置容差时确认为该出生点；大于容差时仍然选择最近的一个，但标记为不确定并记录警告。

等待舰船位置的最长时间为 15 秒。超时后如果配置了默认出生点，使用默认出生点对应的航线；没有默认值时进入异常恢复。出生点只在战斗开始时识别一次，之后不再重复。

两个出生点距离过近时，最近距离可能选错。校验阶段会对这类出生点给出警告，标定人员应当调整出生点坐标或删除歧义较大的点位。

5.9 死亡判定

死亡判定使用多个信号加权投票，避免单一信号误判。每个信号有独立的检测方法和稳定时间要求，综合得分超过阈值且持续时间足够时才判定死亡。

信号	检测方法	权重	稳定时间	说明
血条消失	hp_bar_present 特征连续未命中	0.35	1.0 秒	死亡后己方血条区域消失或被替换
血量为零	血条填充区域颜色比例接近 0	0.25	1.0 秒	血条仍显示但填充为空
死亡提示	death_banner 模板命中	0.25	1.0 秒	提示文字可能只显示数秒
自身标记消失	小地图标记连续未命中	0.10	3.0 秒	可能与遮挡或识别失败混淆
观战界面	spectator_indicator 特征命中	0.05	1.0 秒	确认后作为辅助证据

判定规则：综合得分达到 0.75 且连续稳定 1.0 秒（约连续两帧），判定死亡；血条消失稳定 1.5 秒，或者血条存在但填充接近零稳定 1.5 秒，也直接判定死亡。任意一个强信号出现矛盾时立即清零投票，重新计时。

权重表和阈值是初始值。实机验证时如果发现死亡后血条仍然显示，例如观战界面保留了血条区域，把观战界面信号的权重提高到 0.45～0.5，并增加镜头目标切换特征作为新的死亡信号；如果死亡提示只显示很短时间，降低它的稳定时间要求。调整后必须用误判和漏判样本重新回归，不能只看单场结果。

战斗开始后的前 5 秒不判定死亡，避免加载残留画面误判。死亡判定只在已经确认进入战斗的上下文中运行，判断依据是小地图或战斗 HUD 至少有一个可见。如果连小地图也消失，先回到状态识别，不把“画面里没有血条”当作死亡。

判定死亡后保存一张截图，记录各信号的得分和稳定时间，停止导航和随机行为，进入退出流程。误判为死亡的代价是提前退出，仍然可以开下一场；漏判的代价是脚本一直停在战斗里，因此判定阈值偏向于及时发现死亡，同时用多信号投票控制误判。

如果舰船仍然存活但战斗已经结束，画面会出现结算界面。战斗状态同时检测 `battle_results` 特征，命中后转入战斗结束流程，不再等待死亡信号。这个分支覆盖全队胜利、失败和战斗超时结束的情况。

5.10 弹窗识别与处理

回港后可能出现奖励、任务、每日首胜等弹窗。每个客户端配置提供弹窗特征列表，包括弹窗面板、关闭按钮和确认按钮。处理流程循环执行：识别已知弹窗，点击关闭按钮，等待弹窗消失，最多重复五次。

未知模态窗口出现时，先用 ESC 尝试关闭一次，再重新识别状态。连续两次无法关闭时进入异常恢复，不盲目点击屏幕中心，避免误触其他按钮。

弹窗处理设置总时限，默认 60 秒。超时后记录截图和状态，进入异常恢复。

5.11 感知性能预算

任务	频率	单次预算	说明
整窗截屏	2 Hz	10～20 ms	2560×1600，GDI BitBlt
区域裁剪	2 Hz	小于 2 ms	小地图、血条和提示区域从同一帧裁剪
单特征模板匹配	按需	1～5 ms	搜索区域小，缩放版本已缓存
地图识别	30 秒一次	10～30 ms	ORB 特征与仿射估计
舰船标记识别	2 Hz	5～15 ms	多角度模板匹配，可限制在上一帧附近
死亡信号检测	2 Hz	3～10 ms	血条区域与提示模板

主循环按 10 Hz 设计，截屏按 2 Hz 执行，每 5 轮刷新一帧，其余轮次复用最近一帧。每轮决策预算 100 毫秒，截屏帧预算 300 毫秒，2 Hz 的间隔为 500 毫秒，留有足够余量。运行日志记录每类任务的平均耗时和第 95 百分位耗时，用于后续调优。

6 状态机详细设计

6.1 状态定义

状态机在初步设计的十一个状态基础上增加 INIT、BATTLE_FINISHED、PAUSED、STOPPING 四个状态，用于处理程序启动、战斗正常结束、安全暂停和退出运行。新增状态不改变主流程，只是把原本混在异常处理里的情况单独建模。

状态	进入动作	离开条件	超时
INIT	加载配置、检查窗口、检查截屏与输入	识别到港口	30 秒
PORT	确认港口界面，确认配置船只和空装备，准备开局	开始步骤完成，进入排队	60 秒

状态	进入动作	离开条件	超时
START_COOP	选择联合作战并点击开始	识别到排队界面	30 秒
QUEUEING	等待匹配	识别到加载界面	600 秒
LOADING	等待战斗 HUD	识别到战斗 HUD	300 秒
BATTLE_ACTIVE	识别地图与出生点，启动导航、随机行为、死亡监测	死亡判定或结算界面	无固定上限，受会话限制
BATTLE_FINISHED	停止子流程，等待结算界面稳定	结算界面可操作	60 秒
DEAD_PENDING_EXIT	停止子流程，短暂等待	打开退出菜单	10 秒
EXIT_MENU_OPEN	定位并点击返回港口	出现确认框或进入退出等待	5 秒
EXIT_CONFIRMING	点击确认按钮	对话框消失	5 秒
EXITING	等待回港	识别到港口或回港弹窗	180 秒
POST_EXIT_INTERFACE	清理弹窗	弹窗清空并回到港口	60 秒
ERROR_RECOVERY	按恢复阶梯尝试回到已知状态	回到港口或进入暂停	30 秒
PAUSED	释放所有输入，等待人工处理	人工恢复或停止	无
STOPPING	停止子流程，写会话结束记录，释放输入	进程退出	10 秒

BATTLE_FINISHED 处理舰船仍存活但战斗结束的情况，包括全队胜利、失败和超时。没有这个状态时，脚本会一直等待死亡信号，停在结算界面无法继续。

PAUSED 是安全状态，所有输入停止，程序不自动恢复。进入条件包括：连续异常恢复失败、游戏进程消失、窗口长时间失焦、看门狗检测到主循环卡死、紧急停止热键被按下。恢复需要人工确认，避免脚本在异常条件下继续操作。

6.2 主转移表

状态机使用显式转移表实现，每条转移包含当前状态、事件或条件、下一状态和进入动作。转移按顺序匹配，第一个满足条件的规则生效。

当前状态	事件或条件	下一状态	动作
INIT	识别到港口	PORT	记录会话开始
INIT	窗口或截屏不可用	PAUSED	记录错误，提示人工处理
PORT	开始步骤全部成功	START_COOP	提交点击动作
START_COOP	识别到排队界面	QUEUEING	写状态事件

当前状态	事件或条件	下一状态	动作
START_COOP	超时未进入排队	ERROR_RECOVERY	记录失败步骤
QUEUEING	识别到加载界面	LOADING	写状态事件
QUEUEING	超时	ERROR_RECOVERY	尝试取消排队或回港
LOADING	识别到战斗 HUD	BATTLE_ACTIVE	创建战斗记录，启动子流程
LOADING	超时	ERROR_RECOVERY	检查是否仍在加载或断线
BATTLE_ACTIVE	死亡判定成立	DEAD_PENDING_EXIT	停止子流程，保存截图
BATTLE_ACTIVE	识别到结算界面	BATTLE_FINISHED	停止子流程
BATTLE_ACTIVE	窗口失去焦点且配置为暂停	PAUSED	释放输入
BATTLE_FINISHED	结算界面可操作	POST_EXIT_INTERFACE	执行结算后清理步骤
DEAD_PENDING_EXIT	退出菜单特征命中	EXIT_MENU_OPEN	记录死亡到退出菜单耗时
DEAD_PENDING_EXIT	超时未打开菜单	ERROR_RECOVERY	重试 ESC
EXIT_MENU_OPEN	点击返回港口	EXIT_CONFIRMING	提交点击
EXIT_MENU_OPEN	没有确认框，直接进入退出等待	EXITING	提交点击
EXIT_CONFIRMING	确认框消失	EXITING	提交确认点击
EXITING	识别到港口	POST_EXIT_INTERFACE	记录回港时间
EXITING	识别到回港弹窗	POST_EXIT_INTERFACE	开始清理弹窗
EXITING	超时	ERROR_RECOVERY	检查游戏是否崩溃或断线
POST_EXIT_INTERFACE	弹窗清空且港口可见	PORT	结束本场记录，检查会话限制
POST_EXIT_INTERFACE	超时	ERROR_RECOVERY	尝试 ESC 和关闭弹窗
ERROR_RECOVERY	恢复成功	PORT	记录恢复结果
ERROR_RECOVERY	恢复失败或次数超限	PAUSED	释放输入，提示人工处理

当前状态	事件或条件	下一状态	动作
PORT	配置船只或空装备校验失败	PAUSED	保存截图，等待人工处理
PORT	会话限制到达	STOPPING	写停止原因
任意状态	紧急停止热键	STOPPING	立即释放输入
任意状态	主循环心跳超时	PAUSED	看门狗释放输入

6.3 战斗内子流程

进入 BATTLE_ACTIVE 时创建三个子流程：导航、随机行为、死亡监测。每轮主循环按固定顺序调用它们，子流程只返回动作，不直接执行。

- 导航子流程：在识别地图、出生点和航线后启动。每轮检查当前航点、舰船位置和移动状态，决定是否点击、重试或跳过。
- 随机行为子流程：独立计时，生成视角动作和攻击动作。攻击动作在开火与消耗品之间二选一，开火占明显多数。行为动作和导航动作都进入同一个输入队列。
- 死亡监测：每轮汇总死亡信号，满足条件时向状态机发出死亡事件。

进入 DEAD_PENDING_EXIT、BATTLE_FINISHED、PAUSED 或 STOPPING 时，三个子流程立即销毁，清空待执行动作。导航子流程停止后不再点击小地图，随机行为停止后不再开火或按消耗品。

子流程的启动有先后顺序：先识别地图和出生点，成功后才启动导航；随机行为在战斗 HUD 稳定后启动；死亡监测从进入战斗就开始运行。地图识别失败时导航不启动，状态机记录警告并继续监测死亡，不在未知地图上盲目点击。

6.4 异常恢复

异常恢复按固定阶梯执行，每一步失败后才进入下一步。阶梯设计的原则是从低风险操作开始，避免在状态不明时连续点击。

- 第一步：重新截屏并识别状态。能识别出已知状态就直接恢复。
- 第二步：尝试关闭已知弹窗，最多三次。
- 第三步：按一次 ESC，识别是否出现退出菜单或系统菜单，不连续按键。
- 第四步：如果退出菜单出现，点击返回港口；否则等待并重新识别。
- 第五步：30 秒内仍无法恢复，进入 PAUSED，记录截图和最近状态轨迹，等待人工处理。

异常恢复记录连续失败次数。10 分钟内进入异常恢复超过三次时，直接进入 PAUSED，不再自动重试。频繁恢复通常说明界面模板或坐标配置有误，继续运行只会重复失败。

6.5 状态机实现方式

状态机不使用第三方框架，使用显式的状态处理器和转移表实现。每个状态对应一个处理函数，处理函数接收观察结果和当前时间，返回动作列表和可能的转移请求。转移表集中定义合法转移，非法转移写入错误日志并进入异常恢复。

状态机的决策函数不直接调用截屏、键鼠或数据库，所有输入通过参数传入，所有输出通过返回值传出。这样可以在单元测试中用假观察和假时钟覆盖全部转移，不需要启动游戏。

状态机不持久化到磁盘。程序重新启动后总是从 INIT 开始，识别当前界面后进入对应状态。战斗中途崩溃时本场记录标记为异常结束，不尝试从中断处恢复。

6.6 典型时序

正常一场战斗的时序：港口识别成功，确认配置船只和空装备，执行开局步骤，进入排队；匹配成功后进入加载；加载完成识别到 HUD，识别地图和出生点，选择航线；导航点击逐个航点，随机行为按计时触发；被击沉后判定死亡，打开 ESC 菜单，点击返回港口，清理弹窗，回到港口；检查会话限制，未到达上限则等待随机间隔后开始下一场。

退出菜单没有确认框的时序：点击返回港口后，界面直接进入退出等待，EXIT_CONFIRMING 的确认步骤标记为可选，未命中时跳过，直接等待港口。

回港后有未知弹窗的时序：识别到弹窗但不在已知列表中，按一次 ESC，重新识别；弹窗消失则回到港口，仍存在则进入异常恢复。

导航卡住的时序：15 秒内位移小于 80 米，重新点击当前航点；两次重试仍无位移，跳过当前航点；跳过三个航点仍卡住，停止导航，保留随机行为，等待死亡或战斗结束。

6.7 会话与场次控制

脚本以会话为单位运行，一个会话包含若干场战斗。会话开始写入一条会话记录，记录客户端配置、配置摘要和开始时间；会话结束记录停止原因和总场次。

两场战斗之间等待 15～60 秒的随机时间，避免固定节奏。会话达到最大场次、最大时长、每日场次上限或允许运行时段结束时，进入 STOPPING，不再开始下一场。

每日场次上限跨进程累计，从数据库查询当天已完成的场次。上限的作用是降低连续挂机带来的账号风险，具体数值由使用者配置。

停止原因包括：达到场次上限、达到时长上限、达到每日上限、超出允许时段、紧急停止、连续异常、窗口丢失、游戏崩溃。停止原因写入会话记录，方便事后判断脚本为什么停下。

6.8 船只与装备确认

每次会话固定使用一艘船，船只在启动时通过命令行参数 `--ship` 或全局配置 `ship.ship_id` 指定，命令行优先。会话运行期间不切换船只，也不使用信号旗、经济加成卡或其他消耗性加成。

船只数据放在客户端配置下，每艘船一个目录：`profiles/<client_id>/ships/<ship_id>/`。`ship.yaml` 记录显示名称、卡片模板和装备校验配置，`card.png` 是从港口截取的船只卡片，`loadout/` 存放空信号旗槽和空加成槽的校验模板。

字段	类型	说明
ship_id	字符串	船只编号，必须与目录名一致
display_name	字符串	船只显示名称，仅用于日志和标定界面
card_template	字符串	港口船只卡片模板路径，默认 card.png
card_search_region	四个数字	船只列表的归一化搜索区域，可留空使用客户端配置
loadout_preset_feature	字符串或 null	空装备预设的特征名，有预设时优先选择预设
max_scrolls	整数	船只列表最多滚动次数，默认 20
notes	字符串	涂装、截图条件等备注

进入港口后先确认当前船只。读取港口界面的船只指示特征和卡片模板，如果与配置船只一致，直接进入装备检查；不一致时执行 `ship_selection_sequence`：打开船只列表，按配置方向滚动，用卡片模板匹配目标船只，找到后点击卡片，再确认港口标题或选中卡片已经变为目标船只。

船只列表最多滚动配置的次数。滚动到底仍未找到目标船只时进入 PAUSED，保存截图，不点击列表中当前显示的其他船只。点错船只会让脚本用错误的船进入战斗，后果比暂停严重。

装备检查先看是否存在空装备预设。配置了 `loadout_preset_feature` 时，点击预设并等待选中状态，预设由使用者提前在游戏里创建，内容为不装信号旗、不带经济加成卡。没有预设时，直接检查当前装备。

检查项包括所有信号旗槽位为空、经济加成槽位为空。每个槽位用客户端配置中的空槽模板匹配，全部命中才算通过。任何一项不通过时进入 PAUSED，默认不自动移除已装配的信号旗或加成卡，因为点击这些槽位可能消耗或误操作物品。

如果使用者明确配置了 `clear_loadout_sequence`，脚本可以按步骤点击清除按钮，但该步骤默认关闭，只有实机确认按钮含义和后果后才启用。启用后仍然要在清除完成后重新执行空槽校验。

`ship.verify_each_battle` 为 true 时，每场开始前重新执行一次轻量校验：确认当前船只仍是配置船只，装备槽仍为空。轻量校验只比较特征，不重新滚动船只列表；校验失败时才执行完整的船只选择和装备检查。

船只和装备的校验结果写入状态事件，包含 ship_id、卡片匹配分数、各槽位是否为空和失败原因。校验过程中的截图属于当前场次，战斗结束后按截图规则删除。

7 自动导航详细设计

7.1 导航启动前提

导航只在以下条件全部满足时启动：当前处于 BATTLE_ACTIVE；地图识别成功；出生点识别成功；找到绑定该出生点的航线；小地图标定可用；自身舰船位置可用。任一条件不满足时导航不启动，记录原因，状态机继续监测死亡和战斗结束。

导航器保存一份内部状态，包括当前航线、当前航点序号、上一次点击时间、重试次数、连续卡住次数、最后确认位置和速度估计。进入战斗时创建，退出战斗时销毁，不跨场复用。

```
@dataclass
class NavigatorState:
    route_id: str
    waypoints: list[tuple[float, float]]   # 世界坐标
    index: int = 0
    last_click_at: float = 0.0
    last_click_target: tuple[float, float] | None = None
    retries: int = 0
    stuck_checks: int = 0
    skipped: int = 0
    abandoned: bool = False
```

7.2 导航点击面

默认在小地图上点击设置自动航行目标。小地图面积小，点击精度受像素密度限制，但操作简单，不需要打开全屏地图，也不会遮挡画面。

以 12 公里见方的地图显示在 300 像素宽的小地图上计算，每个像素约对应 40 米。航点到达判定设置 400 米容差，可以吸收约 10 个像素的点击误差，足够覆盖操作误差和小地图缩放误差。

全屏地图模式作为备选，由 `navigation.surface` 配置开启。流程是打开全屏地图、点击目标、关闭地图。全屏地图点击精度更高，但多了一步打开和关闭，地图界面可能暂停其他操作，出错时还会把界面留在打开状态。第一版默认不用，等小地图模式验证稳定后再评估。

两种模式共用同一套坐标变换，只是目标区域从小地图内容区换成全屏地图区域。全屏地图的归一化区域、边框和开关按键放在客户端配置中。

7.3 导航主循环

导航器每轮根据舰船位置、目标航点和时间决定下一步动作。核心规则是：一次只设置一个目标，点击后确认舰船确实移动，到达后切换到下一个航点。

每轮执行：
1. 如果导航已放弃或航线已结束，返回空动作
2. 读取舰船位置 `P` 和速度 `v`；位置不可用时执行降级逻辑
3. 如果当前航点已到达，序号加一，重置点击与重试状态
4. 如果没有待确认的点击：
   `target = waypoints[index]`
   `screen = world_to_screen(target)`
   提交绝对点击动作
   记录 `last_click_at`、`last_click_target`、点击时的位置
5. 如果距离点击已超过确认窗口（首次 `8` 秒，之后 `5` 秒）且尚未确认移动：
   如果位移 `>= move_confirm_m`，视为点击生效
   否则重试点击；重试次数超过上限则跳过当前航点
6. 如果距离目标 `<= arrival_m`，视为到达
7. 如果连续卡住次数超过上限，放弃导航并记录

点击后不连续重发。每次点击会重置自动航行目标，频繁点击会让舰船反复调整方向。确认窗口默认 5 秒，确认位移默认 20 米。舰船从静止起步时加速度有限，5 秒内位移可能不足，因此第一次点击后的确认窗口延长到 8 秒。

自身舰船位置更新频率与截屏一致，为每秒 2 帧。速度估计使用最近 4～6 秒的位移，也就是 8～12 个位置样本，避免单帧抖动。位置置信度低于阈值时不把位移不足当作点击失败，也不把位置跳变当作卡住，只等待下一次有效位置。

7.4 移动确认与速度估计

移动确认使用位移而不是航速读数。点击后记录舰船位置和时间，经过确认窗口后计算直线距离。距离超过 20 米视为已经移动，低于 20 米视为未移动。舰船正在转向时直线位移会小于航迹长度，因此阈值不能设得太高。

速度估计使用最近 3～5 秒内多次位置的位移除以时间，取中位数，减少识别抖动。速度用于判断卡住和预测位置，不用于控制油门。脚本不操作油门，航速由游戏自动航行控制。

舰船位置不可用时，导航进入降级模式：保留当前目标，不发送新点击，等待位置恢复。连续 10 秒无法恢复时放弃导航并记录，避免在没有位置反馈的情况下反复点击。

7.5 到达判定

到达判定同时使用距离和越点两个条件。距离目标小于 `arrival_m`（默认 400 米）时判定到达；或者舰船已经越过目标点、且距离小于 800 米时也判定到达。越点判定用目标点前后两个航点构成的线段方向计算投影，适用于舰船高速通过航点的情况。

到达后切换到下一个航点。最后一个航点到达后按配置执行航线结束行为。切换航点时重置重试次数和卡住计数，但不重置整条航线的跳过计数。

7.6 卡住判定与恢复

卡住判定需要同时满足两个条件：在 15 秒观察窗口内舰船位移小于 80 米，且估计速度低于 1 米每秒。只看位移会把转向和掉头误判为卡住，加入速度条件后可以区分。

恢复分三级。第一级重新点击当前航点，最多两次，每次间隔 4 秒。第二级跳过当前航点，直接点击下一个航点；连续跳过三个航点后进入第三级。第三级放弃导航，停止发送点击，保留随机行为，等待被击沉或战斗结束。

卡住可能由岛屿、浅滩、其他舰船阻挡或自动航行路径不可达造成。脚本不模拟 WASD 走位脱困，也不尝试倒车，因为这类操作需要更精确的位置反馈和碰撞判断，超出当前设计范围。

每次卡住、重试、跳过和放弃都写入战斗动作日志，附带位置、目标、距离和速度，便于事后调整航线。放弃导航时保存一张截图。

7.7 航线结束后的行为

航线结束后默认原地等待。脚本停止导航点击，随机行为继续运行，直到死亡或战斗结束。原地等待的风险是舰船停在固定位置，可能被集火，但这符合“尽快结束本场”的目标。

备选行为包括按原路返回和在当前点附近随机选一个等待点。按原路返回会让航迹更有变化，但返回过程中经过的航段已经验证过，安全性较高。随机等待点需要确保落点在水域内，标定成本更高，第一版不实现。

7.8 输入串行化

导航点击和随机行为共用输入队列。导航提交的是绝对坐标点击，随机行为提交的是相对移动、开火点击和按键。输入线程按提交顺序执行，一个动作完成后才执行下一个。

点击动作作为一个原子动作提交，包含鼠标移动到目标、按下、释放三个步骤，中间不插入其他动作。视角移动和点击之间保持至少 150 毫秒间隔，避免点击落在移动过程中的错误位置。

输入队列积压超过 10 个动作时，丢弃最早的非关键行为动作，保留导航点击和退出相关动作。这样即使某一轮卡顿，也不会在恢复后连续发出一串过期操作。

7.9 导航日志

导航记录以下事件：选择航线、启动导航、点击航点、确认移动、到达航点、重试、卡住、跳过、放弃、航线结束。每条记录包含战斗编号、时间、航点序号、世界坐标、屏幕坐标、当前舰船位置、距离和速度。

这些记录用于两类分析：判断航线是否合理，例如某航点反复超时；判断点击坐标是否准确，例如点击后舰船持续向错误方向移动。日志中的坐标都是世界坐标和屏幕坐标成对出现，方便对照标定结果。

8 防挂机行为详细设计

8.1 设计目标与边界

防挂机行为的目标是让客户端持续收到正常的玩家输入，降低被判定为挂机的概率，同时不改变战斗结果，不做任何瞄准或战术决策。行为包括视角随机移动和攻击动作，攻击动作分为开火与点击消耗品两种。

开火和消耗品共用一个攻击计时器，每次攻击事件只执行其中一种，不并列执行。消耗品可能因为冷却、启用条件或数量限制而不产生效果，因此攻击事件以开火为主，消耗品只占少数。

所有行为只在 BATTLE_ACTIVE 状态运行。进入死亡退出、战斗结束、暂停或停止状态后立即全部中止，不补发积压动作。行为不判断敌人位置、不判断弹药类型、不判断血量，只按随机调度产生输入。

8.2 调度器结构

行为调度器维护两个独立计时器，分别对应视角和攻击，另有一个活动看门狗。攻击计时器到期时，按本场随机档案中的 `consumable_ratio` 决定本次执行开火还是消耗品。

```
class BehaviorScheduler:
    cameras: Timer
    attack: Timer
    activity_watchdog: Timer
    profile: RandomProfile
    consumables_used: int = 0

    def tick(now):
        actions = []
        if cameras.due(now):
            actions += make_camera_action()      # 可能包含多次相对移动
            cameras.next = now + sample_camera_interval()
        if attack.due(now):
            use_consumable = (
                random() < profile.consumable_ratio
                and consumables_used < config.consumable_max_per_battle
            )
            if use_consumable:
                actions += make_consumable_action()
                consumables_used += 1
            else:
                actions += make_shoot_action()
            attack.next = now + sample_attack_interval()
        if activity_watchdog.due(now) and idle_too_long():
            actions += make_camera_action()
            activity_watchdog.next = now + sample_watchdog_interval()
        return actions
```

计时器使用单调时钟。动作提交后刷新最后活动时间，任意行为都会重置看门狗。计时器到期时如果检测到弹窗、退出菜单或窗口失焦，跳过本次动作并顺延，不在非战斗界面发送输入。

攻击事件只产生一个动作。消耗品达到单场次数上限后，即使随机数命中消耗品，也改为执行开火，保证攻击节奏不中断。

8.3 本场随机档案

每场战斗开始时生成一份随机档案，决定这场的整体行为倾向。档案使用系统随机源生成种子，之后本场内所有随机数由该种子驱动，保证复盘时可以复现。

字段	取值范围	作用
camera_activity	0.6～1.4	视角动作频率倍数，越大越活跃
attack_frequency	0.5～1.5	攻击事件间隔倍数，越大间隔越短
consumable_ratio	0.05～0.25	攻击事件选择消耗品的概率，其余选择开火
long_stare_bias	0.0～1.0	长时间不动的倾向，越大越容易出现长间隔
seed	32 位整数	本场随机种子，写入数据库

档案只影响行为节奏，不影响导航、死亡判定和退出流程。即使某场行为非常安静，状态机和导航仍按固定规则运行。

`consumable_ratio` 默认 0.15，允许范围 0.05～0.25，对应约 75%～95% 的攻击事件是开火。范围上限限制消耗品占比，保证开火明显多于消耗品。

8.4 视角随机晃动

视角动作的触发间隔服从正态分布，均值为 8 秒乘以 `camera_activity`，标准差 3 秒，结果钳位在 3～15 秒。正态分布让动作有时密集、有时稀疏，避免固定周期。

每次动作选择移动方向，方向在上一方向的基础上加一个随机偏转，形成缓慢变化的视线轨迹，避免每次完全随机导致视角乱跳。移动幅度在 20～160 像素之间，拆成 3～6 个小步，在 0.2～0.8 秒内完成，速度曲线采用缓入缓出。

有一定概率进入长时间凝视：不移动视角，持续 5～20 秒，下一次视角动作的间隔相应延长。概率由 `long_stare_bias` 控制，默认 15%。凝视期间攻击行为照常触发。

视角使用鼠标相对移动实现。实际旋转角度取决于游戏内鼠标灵敏度，配置提供灵敏度倍率，实机标定时调整到“动作可见但不影响观察”的程度。

8.5 攻击事件与开火

攻击事件间隔服从正态分布，均值为 60 秒乘以 `attack_frequency`，标准差 25 秒，钳位在 25～120 秒。触发时生成一个 0～1 的随机数，小于 `consumable_ratio` 时选择消耗品，否则选择开火。

选择开火时，从当前视角前方左右 60 度的扇形范围内选一个方向，有 50% 的概率先转动视角朝该方向，再点击鼠标左键。

每次开火是 1～4 发的点射，每发之间间隔 0.3～1.1 秒。炮塔是否装填完成、是否有目标、是否在射程内都不做判断，未装填时点击不会产生效果，这符合“纯随机输入”的定位。

开火点击落在画面中心的战斗视角区域，不落在小地图、血条或其他 UI 上。点击前检查当前没有弹窗和退出菜单，检查失败时跳过本次攻击事件。

开火可能暴露舰船位置，也可能命中敌舰，这些后果不在设计目标内。脚本不根据命中反馈调整行为，也不使用任何战斗信息。

8.6 消耗品随机点击

消耗品不单独计时，只在攻击事件选中时执行。选中后从配置的消耗品栏位中随机选一个按键按下。

消耗品不做血量、状态或冷却判断。栏位不可用、正在冷却或未满足启用条件时按键没有效果，这是可以接受的，也是消耗品占比低于开火的原因。

单场点击总次数设上限，默认 10 次。达到上限后，攻击事件全部选择开火，不再点击消耗品。

消耗品键位来自客户端按键配置，不假设 1～5 是固定栏位。配置中的每个栏位带可读名称，日志记录名称和实际按键，便于和游戏内设置对照。

8.7 活动看门狗

维护一个最后活动时间，任意行为动作或导航点击都会刷新它。如果当前时间减去最后活动时间超过一个随机区间（默认 35～95 秒），主动触发一次视角动作。

看门狗解决两个问题：航线结束原地等待时长时间没有任何输入；随机计时器因异常被推迟后长时间静默。看门狗的动作只做视角移动，不额外触发攻击，避免在不需要的时候增加可疑操作。

看门狗区间每次触发后重新采样，不固定为 60 秒，也不使用同一个间隔反复触发。

8.8 参数钳位与安全上限

所有采样结果在写入计时器前钳位到配置范围。正态分布可能出现极端值，钳位保证不会出现 1 秒内连续攻击或 10 分钟没有任何动作。

行为模块还有三类硬性限制：战斗开始后前 10 秒不执行行为，等待界面稳定；每分钟动作数超过上限时丢弃多余动作；任何时刻检测到死亡、战斗结束、暂停或窗口失焦，立即停止并清空计时器。

动作频率上限默认每分钟 20 个动作，足以覆盖视角和攻击，又能防止配置错误导致的高频输入。

8.9 行为日志

每次行为动作记录类型、时间、参数和执行结果。视角记录方向、幅度、步数和耗时。攻击事件记录本次选择的是开火还是消耗品、随机数和选择依据；开火记录方向、发数和间隔；消耗品记录栏位名称和按键。日志同时关联本场随机档案的编号，复盘时可以按档案统计开火与消耗品的实际比例。

日志不记录屏幕内容。截图只在当前场次保存，战斗结束后按第 11.6 节的规则删除。

9 输入模块详细设计

9.1 动作模型

输入模块接收统一的动作对象，动作类型包括绝对点击、相对移动、按键按下、按键释放、鼠标按下、鼠标释放和等待。导航和行为模块只构造动作，不直接调用 Windows 接口。

```
@dataclass(frozen=True)
class Action:
    kind: str                # click_abs / move_rel / key / mouse_down / mouse_up / wait
    x: int | None = None     # 屏幕绝对坐标
    y: int | None = None
    dx: int | None = None    # 相对位移
    dy: int | None = None
    key: str | None = None   # 可读按键名
    duration_ms: int = 0
    label: str = ""          # 日志用途，例如 nav_click、camera_move
```

动作队列由输入线程消费。队列中的动作按提交顺序执行，执行结果写回日志。导航点击、开火连发这类由多个底层事件组成的动作，作为一个复合动作提交，中间不插入其他动作。

9.2 SendInput 封装

使用 ctypes 调用 user32.SendInput，定义 INPUT、MOUSEINPUT、KEYBDINPUT 三个结构体。鼠标绝对移动把屏幕坐标归一化到 0～65535，并设置 MOUSEEVENTF_ABSOLUTE 和 MOUSEEVENTF_MOVE；相对移动只设置 MOUSEEVENTF_MOVE，dx、dy 为像素位移。

```
绝对点击（按虚拟屏幕归一化）：
  dx = round((x - vs_left) * 65535 / (vs_width - 1))
  dy = round((y - vs_top) * 65535 / (vs_height - 1))
  flags = MOUSEEVENTF_MOVE | MOUSEEVENTF_ABSOLUTE | MOUSEEVENTF_VIRTUALDESK
  发送左键按下，等待 30～80 ms，发送左键释放

相对移动：
  flags = MOUSEEVENTF_MOVE
  dx, dy = 本次步长
  分多步发送，每步间隔 20～60 ms

按键：
  优先使用扫描码，设置 KEYEVENTF_SCANCODE
  按下后等待 30～60 ms，再发送释放
```

绝对坐标按整个屏幕的虚拟分辨率归一化，不是按游戏窗口。点击前由坐标模块把窗口内坐标加上窗口左上角的屏幕坐标，得到屏幕坐标。多显示器时使用虚拟屏幕的左上角偏移。

每次调用 SendInput 检查返回值。返回值不等于输入事件数量时记录错误，不静默继续。连续失败三次时进入暂停状态，避免在输入不可用时继续运行。

按键映射维护可读名称到虚拟键码和扫描码的对应表。配置中出现不支持的名称时，启动校验直接报错。

9.3 窗口焦点管理

脚本启动时按进程名查找游戏窗口，找不到时按窗口标题匹配。取得窗口句柄后读取客户区尺寸和屏幕位置，窗口尺寸变化时重新读取。

点击前检查游戏窗口是否为前台窗口。不是前台时，根据配置决定等待还是暂停：pause_when_unfocused 为 true 时进入 PAUSED，为 false 时尝试把游戏切到前台。切换前台

使用 SetForegroundWindow，必要时用 AttachThreadInput 连接输入线程，避免系统拒绝切换。

窗口最小化、客户区尺寸为零或窗口句柄失效时，视为游戏不可用，进入异常恢复或暂停。脚本不尝试恢复最小化窗口，也不启动游戏进程。

可选配置把游戏窗口置顶，防止其他窗口遮挡影响截屏。置顶会改变桌面使用体验，默认关闭，只在无人值守运行时开启。

9.4 输入线程与节奏

输入线程是唯一的键鼠出口。主线程把动作放入线程安全队列，输入线程取出后执行，执行完记录结果。队列使用阻塞队列，主线程不会因为输入慢而阻塞太久。

两个动作之间保持最小间隔 150 毫秒，点击的按下与释放间隔 30～80 毫秒，按键的按下与释放间隔 30～60 毫秒。间隔加入 ±20% 的随机抖动，避免完全固定的节奏。

紧急停止时清空队列，释放所有可能处于按下状态的键和鼠标按钮，然后设置停止标志。输入线程在每次执行动作前检查停止标志，标志有效时立即返回。

9.5 安全控制

紧急停止热键默认 F12，可在全局配置中修改。看门狗线程以 20 Hz 轮询 GetAsyncKeyState，检测到热键按下时立即触发停止，不等待主循环。

其他安全控制包括：单次会话最长运行时间；单次会话最大场次；每日场次上限；主循环心跳超时；窗口长时间失焦。任一条件触发时，程序先释放输入，再进入 PAUSED 或 STOPPING。

程序退出时遍历所有按下的键和鼠标按钮，发送释放事件。即使异常退出，也会在退出处理函数中尝试释放，避免游戏里出现按键卡住。

9.6 兼容性验证与备选实现

输入方案需要在实机上逐项验证：绝对点击是否能点中小地图、相对移动是否旋转视角、鼠标左键是否开火、按键是否触发消耗品。验证结果记录到第 16 章清单。

如果 SendInput 的绝对点击被游戏忽略，备选方案是先用 SetCursorPos 移动光标，再发送鼠标按下和释放。如果相对移动无效，备选方案是调整鼠标灵敏度或改用绝对移动模拟视角。输入模块保留后端接口，两种实现可以通过配置切换，不影响上层模块。

输入后端接口定义为 `send(actions)` 和 `release_all()`，上层只依赖这两个方法。切换实现时不修改导航和行为代码。

9.7 输入日志

每个动作记录类型、提交时间、执行时间、耗时、坐标或按键、标签和执行结果。坐标同时记录窗口内坐标和屏幕坐标，便于核对 DPI 和窗口位置。

日志不记录按键对应的文字内容，只记录动作名称，避免把任何输入内容写入日志。日志文件只保存在本地。

10 退出与续场详细设计

10.1 步骤序列执行器

开局、退出和回港弹窗处理都由同一种步骤序列描述。执行器逐条执行步骤，支持按键、点击界面特征、等待界面特征、等待固定时间四种基本动作。每个步骤可以设置超时、重试次数和是否可选。

步骤类型	参数	行为
key	key、wait_after_ms	发送一次按键，等待指定时间
click_feature	feature、timeout_ms、optional	在超时内反复查找特征，找到后点击特征中心
wait_feature	feature、timeout_ms、optional	在超时内等待特征出现，不点击
wait_ms	duration_ms	等待固定时间，用于界面动画

步骤失败的处理顺序是：重试当前步骤，重试次数用完后如果步骤标记为可选则跳过，否则整个序列失败并进入异常恢复。执行器记录每一步的开始时间、耗时和结果，写入状态事件。

步骤序列完全由客户端配置描述，代码中没有针对某个客户端的退出分支。客户端更新界面后，只需要重新截取模板和调整步骤，不需要改代码。

10.2 死亡后的退出流程

死亡判定成立后，先等待 0.8～2.0 秒的随机时间，让死亡动画和界面变化稳定，再执行退出序列。等待时间过短时，ESC 菜单可能还没有响应；过长会浪费场次时间。

典型退出序列是：按 ESC 打开系统菜单，查找返回港口按钮，点击按钮，如果出现确认框则点击确认，等待港口界面出现。具体按键和按钮位置由客户端配置决定。

死亡后的视角可能因死亡方式不同而变化：自己舰船沉没、观战队友、全队失败。设计假设这几种情况下 ESC 菜单和返回港口入口一致，实机验证时分别检查。如果某个视角下入口不同，在客户端配置中增加可选步骤，执行器按顺序尝试。

部分版本可能在死亡后提供直接的离开战斗按钮，不经过 ESC 菜单。配置可以把该按钮作为可选的第一步，命中则跳过后续步骤。没有命中时按常规 ESC 路径执行。

退出方式记录到战斗记录中，包括菜单退出、确认框退出、直接退出和异常退出四种取值，便于统计不同路径的成功率。

从死亡判定到点击返回港口的耗时单独记录。这个耗时直接影响每场的周转时间，是后续优化的重点指标。

10.3 战斗结束时仍存活的流程

舰船存活但战斗结束，画面出现结算界面。感知模块检测到结算特征后，状态机进入 BATTLE_FINISHED，停止导航和随机行为。

结算界面通常自动播放奖励和经验，随后出现返回港口按钮。执行器等待结算按钮可用，点击返回港口，再进入回港弹窗处理。结算界面持续时间和按钮位置由客户端配置描述，无法确认时按 ESC 尝试，再进入异常恢复。

10.4 返回港口确认

回港确认使用港口主界面的特征，例如战斗按钮、港口背景标志或模式选择器。只匹配一个特征容易误判，设计使用两个特征同时命中，或者主特征连续两帧命中。

确认回港后，先检查游戏进程仍然存在。进程消失说明游戏崩溃或退出，进入异常恢复的进程检查分支，而不是继续等待港口界面。

从点击返回港口到确认回港的耗时写入战斗记录。超时未回港时保存截图，记录当前识别到的状态，进入异常恢复。

10.5 回港弹窗清理

回到港口后可能出现奖励结算、任务完成、每日首胜、活动通知等弹窗。清理流程循环执行：识别已知弹窗，点击关闭或确认按钮，等待弹窗消失，再次识别，最多五轮。

每一轮之间等待 300～800 毫秒，给界面动画留出时间。点击后特征仍然存在时，视为点击未生效，换用备用关闭特征或按 ESC。

清理流程总时限 60 秒。超时或遇到未知弹窗时，保存截图，进入异常恢复。弹窗清理失败不应阻止下一场，但也不能在弹窗遮挡下盲目点击开始战斗。

10.6 进入下一场

回港并清理弹窗后，等待 15～60 秒的随机间隔，再开始下一场。间隔的作用是避免固定节奏，也给玩家和服务器一个自然的停顿。

开局前确认当前船只仍是配置船只，信号旗槽和经济加成槽为空。配置开启每场校验时，轻量校验每场执行一次；校验失败时执行完整的船只选择和装备检查，仍失败则进入 PAUSED，不开始战斗。

开局优先使用快速路径：先确认港口界面显示当前战斗模式为联合作战，再点击开始战斗按钮。游戏通常会记住上一次选择的战斗模式，模式确认无误时快速路径一步就能进入排队。

如果看不到联合作战模式指示，说明当前可能停留在随机战斗或其他模式，直接改用完整路径：打开战斗模式选择，选择联合作战，确认模式指示出现，再点击开始。完整路径步骤更多，但可以避免误入随机战斗队列。

快速路径点击后 10 秒内没有出现排队界面，或者模式指示发生变化，也改用完整路径。两种路径都失败时，记录当前界面截图和状态，进入异常恢复。

连续两场开局失败时进入 PAUSED，提示人工检查港口界面。

10.7 退出与续场失败分支

现象	可能原因	处理
按 ESC 后菜单未出现	界面未响应、按键未送达、焦点丢失	等待 1 秒重试一次，仍失败进入异常恢复
菜单出现但没有返回港口按钮	客户端版本不同、当前视角限制	尝试配置中的备用按钮，再按 ESC 关闭菜单
点击返回港口后无反应	坐标偏差、按钮被遮挡	重新定位特征后点击，最多两次
确认框出现但点击无效	确认按钮模板过期	保存截图，进入异常恢复
等待回港超时	游戏崩溃、断线、卡在加载	检查进程和窗口，进入异常恢复
回港后弹窗无法关闭	新活动弹窗、模板缺失	按 ESC 一次，仍失败进入异常恢复
开始战斗后未排队	模式选错、按钮未点中、队列限制	改用完整开局路径，再失败进入异常恢复

现象	可能原因	处理
船只或装备校验失败	选错船只、装备未清空、模板过期	进入 PAUSED，保存截图，等待人工处理
连续多场开局失败	界面改版或账号状态异常	进入 PAUSED，等待人工处理

10.8 周转时间指标

每场记录三个时间：死亡判定到点击返回港口的间隔、点击返回港口到确认回港的间隔、确认回港到进入下一场排队的间隔。三个指标分别反映退出操作、服务器返回和开局操作的表现。

指标写入战斗记录，并在会话结束时输出平均值和中位数。优化目标是缩短死亡到回港的时间，同时保持退出成功率。

11 数据与日志详细设计

11.1 数据库概况

SQLite 数据库只保存运行日志和会话记录，不保存地图配置。数据库文件默认位于 `data/wows_script.db`，启用 WAL 模式，允许日志线程写入的同时用只读连接查询。连接初始化时执行 `PRAGMA foreign_keys = ON` 和 `PRAGMA journal_mode = WAL`。

数据库只有日志线程写入，其他线程通过事件队列提交记录。写入使用批量事务，每秒最多提交一次，减少磁盘同步次数。写入失败时降级为文本日志，不中断脚本运行。

时间字段统一使用带时区的 ISO-8601 文本，例如 `2026-10-30T18:23:00+08:00`。同一时区内的文本排序等价于时间排序，查询和人工阅读都方便。

11.2 表结构

```
CREATE TABLE sessions (
    session_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    started_at      TEXT NOT NULL,
    ended_at        TEXT,
    profile_id      TEXT NOT NULL,
    config_hash     TEXT NOT NULL,
    stop_reason     TEXT
);

CREATE TABLE battles (
    battle_id             INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id            INTEGER NOT NULL REFERENCES sessions(session_id),
    started_at            TEXT NOT NULL,
    map_id                TEXT,
```

```
    spawn_id              TEXT,
    route_id              TEXT,
    death_detected_at     TEXT,
    exit_requested_at     TEXT,
    exit_confirmed_at     TEXT,
    returned_to_port_at   TEXT,
    exit_method           TEXT,
    exit_error            TEXT,
    random_profile_id     INTEGER,
    result                TEXT
);

CREATE TABLE battle_random_profiles (
    profile_id             INTEGER PRIMARY KEY AUTOINCREMENT,
    battle_id              INTEGER REFERENCES battles(battle_id),
    camera_activity        REAL NOT NULL,
    attack_frequency       REAL NOT NULL,
    consumable_ratio       REAL NOT NULL,
    long_stare_bias        REAL NOT NULL,
    seed                   INTEGER NOT NULL
);

CREATE TABLE battle_actions (
    action_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    battle_id          INTEGER REFERENCES battles(battle_id),
    occurred_at        TEXT NOT NULL,
    action_type        TEXT NOT NULL,
    detail             TEXT,
    random_profile_id  INTEGER
);

CREATE TABLE state_events (
    event_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id   INTEGER REFERENCES sessions(session_id),
    battle_id    INTEGER REFERENCES battles(battle_id),
    occurred_at  TEXT NOT NULL,
    state        TEXT NOT NULL,
    confidence   REAL,
    detail       TEXT
);
```

battles.result 记录本场结束方式，取值为 dead、finished、aborted。exit_method 取值为 menu、confirm、direct、recovery、none。exit_error 保存退出失败的原因，成功时为空。sessions.config_hash 是运行配置内容的 SHA-256 摘要前 16 位，用于区分不同配置下的运行结果。

battle_random_profiles 与 battles 通过 random_profile_id 关联。为避免循环外键，先插入战斗记录，再插入随机档案，最后更新战斗记录的 random_profile_id。

11.3 索引与常用查询

```
CREATE INDEX idx_battles_started ON battles(started_at);
CREATE INDEX idx_actions_battle_time ON battle_actions(battle_id, occurred_at);
CREATE INDEX idx_states_battle_time ON state_events(battle_id, occurred_at);
```

定位卡死时，按 battle_id 查询 state_events 并按时间排序，可以看到脚本停在哪个状态、最后一次识别置信度是多少。再查询同一场次的 battle_actions，可以看到最后一次点击、按键或恢复动作。

统计退出效率时，对 battles 表计算 death_detected_at 到 exit_requested_at、exit_requested_at 到 returned_to_port_at 的差值。时间文本统一时区，可以直接用 SQLite 的日期函数计算。

每日场次上限通过查询当天 battles 数量实现，按 started_at 的日期部分统计，不依赖内存计数，进程重启后仍然有效。

11.4 动作类型

action_type	含义	detail 主要内容
state_change	状态变化	旧状态、新状态、置信度
nav_select_route	选择航线	route_id、spawn_id
nav_click_waypoint	点击航点	序号、世界坐标、屏幕坐标
nav_waypoint_reached	到达航点	序号、距离
nav_retry	重试点击	序号、重试次数、原因
nav_stuck	判定卡住	位置、速度、观察窗口
nav_skip	跳过航点	序号、原因
nav_abandon	放弃导航	原因、连续卡住次数
camera_move	视角移动	方向、幅度、步数、耗时
attack_choice	攻击事件选择	随机数、选择结果、消耗品已用次数
fire_burst	开火点射	方向、发数、间隔
consumable	消耗品点击	栏位名称、按键
ship_verify	船只校验	ship_id、卡片匹配分数、结果
loadout_verify	装备校验	各槽位是否为空、失败原因
death_detected	判定死亡	各信号得分、稳定时间
exit_step	退出步骤	步骤类型、结果、耗时
popup_dismiss	关闭弹窗	弹窗特征、结果
recovery	异常恢复	尝试的阶梯步骤、结果
session_stop	会话停止	停止原因

action_type	含义	detail 主要内容
capture_error	截屏失败	错误类型、窗口状态
input_error	输入失败	动作类型、SendInput 返回值

11.5 文本日志

文本日志使用 Python 标准 logging，输出到 `logs/session_<开始时间>.log`，同时输出到控制台。日志级别默认为 INFO，记录状态变化、动作、警告和错误；DEBUG 级别记录每次感知结果和输入细节，默认关闭。

每条日志包含时间、级别、模块、会话编号和战斗编号。日志文件按大小轮转，单文件 20 MB，保留最近 5 个文件。文本日志和数据库互补，数据库用于查询，文本日志用于快速定位异常。

11.6 截图与诊断产物

截图保存在 `screenshots/<session>/<battle>/` 目录，文件名包含时间和类型，例如 `20261030_182301_death.png`。保存时机包括：死亡判定、进入异常恢复、导航放弃、地图识别失败、船只或装备校验失败、弹窗处理超时、用户手动触发。

截图只用于当前场次的诊断。战斗结束、确认回到港口或异常终止后，程序删除该场次的截图目录，不长期保留。数据库不记录截图路径，避免出现指向已删除文件的悬空记录；截图事件写入对应的状态事件明细。

进程崩溃可能留下未删除的截图目录。下一次会话启动时，程序扫描 `screenshots/`，删除不属于当前会话的目录，并记录清理结果。

标定工具生成的地图包、船只卡片和界面模板不放在截图目录，按地图、客户端和船只目录保存，不受截图清理影响。录制回放使用的测试画面也是独立数据，由测试人员手动管理。

11.7 数据边界

所有数据保存在本机，不上传、不共享。数据库不记录账号信息、密码、令牌或聊天内容。窗口标题可能包含玩家名称，日志中只记录窗口标题的匹配结果，不保存完整标题。

日志和当前场次截图包含游戏画面，可能包含其他玩家名称。分享日志或截图前需要自行检查，设计不提供自动脱敏功能。

11.8 写入失败处理

数据库写入失败时，日志线程把事件写入备用文本文件 `logs/db_fallback.log`，并记录一次警告。数据库连接每 30 秒尝试重建，重建成功后继续写入。

磁盘空间不足时，先停止保存截图，再降低日志级别，最后进入 PAUSED。脚本不在磁盘写满的情况下继续运行，避免日志丢失后无法定位问题。

12 标定与验证工具详细设计

12.1 工具清单

命令	用途	输出
validate	校验全局配置、客户端配置、地图包	错误和警告列表，退出码
check_env	检查窗口、截屏、DPI、输入可用性	检查报告和测试截图
calibrate-map	标定底图范围、出生点和航线	map.yaml、spawns.yaml、routes.yaml
calibrate-profile	标定小地图区域和界面特征模板	profile.yaml 和 features 图片
calibrate-ship	截取船只卡片、空信号旗槽和空加成槽模板	ships/<ship_id>/ 下的 ship.yaml 和图片
capture-marker	截取自身舰船标记的多角度模板	marker_*.png
record	按固定频率录制画面和元数据	frames 目录和 metadata.json
replay	离线回放录制画面，运行感知逻辑	识别结果和错误统计
stats	统计历史场次、退出耗时和失败原因	控制台报告

12.2 地图标定流程

地图标定使用 tkinter 图形界面，离线运行，不需要游戏在线。界面上方显示底图，下方是字段和按钮。底图可以缩放和拖动，坐标以原始图片像素为准。

- 第一步：从 wowsdb.com 地图库下载底图，保存为 background.png；载入底图和地图元数据，填写地图编号、名称和尺寸。
- 第二步：在底图上拖动矩形，标出可游玩区域，得到 image_bounds。填写 world_bounds，并选择底图方向（正常、上下颠倒、左右镜像或 90 度旋转），界面实时显示米与像素的换算和方向预览。
- 第三步：在底图上点击添加出生点，输入编号、朝向和说明。

- 第四步：选择一个出生点，依次点击添加航点，形成一条航线。可以添加多条航线和调整权重。
- 第五步：打开预览，检查所有出生点和航点是否落在预期位置，航线是否避开岛屿。
- 第六步：保存并校验。校验通过后写入地图包目录。

标定工具保存时自动执行第 4.7 节的校验规则。有错误时阻止保存并高亮问题点，有警告时允许保存但显示提示。

工具支持撤销和重新编辑，配置文件保存为 YAML，不写二进制格式。手工修改 YAML 后可以重新载入继续编辑。

方向选择错误会让所有出生点和航点镜像或颠倒。标定时先用游戏内两个已知位置对照预览，确认方向无误后再录入出生点和航点。地图识别会自动判断方向，声明值与识别值不一致时以识别结果为准。

12.3 客户端配置标定流程

客户端标定需要实机截图。先截取 2560×1600 的港口、排队、加载、战斗、死亡、退出菜单、确认框和回港弹窗画面，保存到临时目录。

- 窗口信息：确认进程名和窗口标题，填入 profile.yaml。
- 小地图区域：在战斗截图上拖动矩形，标出小地图外框和内容区，保存为归一化坐标。
- 界面特征：逐个框选联合作战入口、开始按钮、排队标志、加载标志、战斗 HUD、死亡提示、退出菜单、确认按钮、弹窗关闭按钮，保存为 PNG 模板。
- 步骤序列：按实际操作顺序填写船只选择、装备检查、开局、退出和弹窗步骤，选择对应的特征。
- 舰船标记：从战斗截图中截取自身舰船标记，工具自动旋转生成 24 个角度模板。
- 船只与装备：从港口截图截取目标船只卡片、空信号旗槽和空经济加成槽，保存到 `profiles/<client>/ships/<ship_id>/`，并填写 ship.yaml。

模板截取时尽量包含周围的固定背景，不要只截按钮内部。模板过小或颜色单一容易误匹配，工具会给出尺寸和颜色方差提示。

标定完成后运行 `replay`，用录制的画面检查每个特征是否稳定命中。命中率低于 95% 时重新截取模板或调整搜索区域。

12.4 录制与回放

录制工具按固定频率截取窗口画面，保存 PNG 或压缩的 PNG 序列，同时记录窗口客户区、时间戳和当前手动标注的状态。录制频率默认 2 Hz，一场战斗约几十到上百张图片。

回放工具读取录制画面，按与运行时相同的感知代码逐帧识别，输出识别状态、特征得分和舰船位置。回放不发送任何键鼠事件，可以反复运行。

回放用于三件事：调整模板阈值和搜索区域；验证地图识别和出生点识别；为单元测试提供带标签的画面样本。录制时手动标注的状态作为回放的期望结果，识别不一致的帧保存到 `tests/fixtures/failures` 供分析。

录制和回放使用同一套坐标与窗口元数据，回放时按录制时的窗口尺寸还原归一化区域。窗口尺寸变化时回放结果不可直接比较，需要重新录制。

12.5 环境检查

`check_env` 依次检查：游戏窗口能否找到；客户区尺寸是否与配置一致；DPI 感知是否生效；截屏能否成功且内容不是黑屏；小地图区域能否截取；船只卡片模板能否在港口中匹配；鼠标绝对点击和相对移动是否产生预期效果；紧急停止热键是否可用。

输入检查需要人工配合：工具提示后由使用者观察游戏内是否出现对应反应，再在控制台确认。检查结果写入 `logs/env_check.txt`，包含每项的通过状态、耗时和备注。

环境检查不通过时不进入正式运行。常见问题包括游戏窗口标题变化、以管理员权限运行导致输入被拦截、显示器缩放不是 100%、游戏处于独占全屏。

12.6 实机联调顺序

推荐的联调顺序是：安装游戏并设置为无边框窗口；运行 check_env 确认截屏和输入；截取各类界面截图；完成客户端标定；完成目标船只卡片和空装备的标定；完成一张地图的标定；用 replay 验证识别；手动开始一场战斗，只观察不操作，确认感知和死亡判定；开启导航跑单场；最后开启完整流程连续跑五场。

每一步都保留日志。需要复盘的截图在战斗结束前手动另存，避免被场次结束时的清理删除。前一步没有验证通过时，不进入下一步，避免多个问题叠加后难以定位。

13 异常处理与运行保障

13.1 异常分类

异常	检测方式	处理策略
窗口不存在	按进程名和标题找不到窗口	进入 PAUSED，等待人工处理，可选自动重启
窗口最小化	IsIconic 为真	进入 PAUSED，不尝试恢复窗口
窗口失去焦点	前台窗口不是游戏窗口	按配置暂停或尝试切回前台
截屏失败	BitBlt 返回失败或图像为空	重试两次，仍失败进入异常恢复
截屏黑屏	图像方差接近零	记录警告，暂停截屏，提示可能处于独占全屏
特征未命中	匹配分数低于阈值	连续两秒失败进入异常恢复
地图识别失败	内点数或误差不满足条件	重试三帧，回退加载界面识别，再失败进入异常恢复
船只或装备校验失败	卡片未匹配、槽位非空、模板过期	进入 PAUSED，保存截图，等待人工处理
舰船位置丢失	连续 3 秒未定位	导航降级，10 秒后放弃导航
输入失败	SendInput 返回值不符	重试一次，连续三次进入 PAUSED
数据库写入失败	SQLite 异常	写备用文本日志，30 秒后重建连接
磁盘空间不足	写文件失败或预检查	停止截图，降低日志级别，必要时 PAUSED
游戏崩溃或掉线	进程消失或窗口消失	进入 PAUSED，可选重启游戏
配置错误	启动校验失败	打印错误并退出，不进入主循环

13.2 超时参数

场景	默认超时	配置项	说明
匹配排队	600 秒	timeouts.queue_s	超过后取消排队并回港
战斗加载	300 秒	timeouts.loading_s	超过后检查断线
打开退出菜单	5 秒	timeouts.exit_menu_s	ESC 后等待菜单
退出确认	5 秒	timeouts.exit_confirm_s	确认框出现和消失
退出到港口	180 秒	timeouts.exit_s	包括服务器结算时间
回港后弹窗	60 秒	timeouts.popup_s	清理弹窗总时限
回港确认	180 秒	timeouts.port_s	识别港口界面
异常恢复	30 秒	timeouts.recovery_s	单次恢复尝试
主循环心跳	10 秒	timeouts.heartbeat_s	看门狗判定卡死

超时值都是初始建议，实机验证后按网络和机器性能调整。退出到港口的超时与服务器结算有关，不同客户端和时段可能差异较大，配置留出足够余量。

13.3 游戏崩溃与断线

每轮检查游戏窗口是否仍然存在，进程是否仍然运行。窗口消失或进程退出时，记录当前状态、最近动作和截图，进入 PAUSED。

配置可以提供游戏启动命令和启动器路径，默认不自动重启。开启自动重启时，重启前等待 30 秒，重启后最多尝试两次，仍无法进入港口则进入 PAUSED。自动重启会重新启动客户端，可能触发登录流程，脚本不处理登录和密码输入。

断线通常表现为加载界面长时间停留、战斗界面卡住或返回登录界面。超时后进入异常恢复，恢复失败进入 PAUSED。脚本不自动输入账号密码，也不处理验证码。

13.4 资源限制

程序运行时的资源目标：CPU 平均占用不超过单核的 30%，内存不超过 500 MB，日志磁盘增量每天不超过 100 MB。截图在战斗结束后删除，不长期占用磁盘。超过目标时降低截屏频率、关闭调试日志或减少截图保存。

启动时检查磁盘剩余空间，低于 2 GB 时只允许文本日志，不保存截图；低于 500 MB 时进入 PAUSED。

内存使用在长时间运行后可能因图像缓存增长而上升。模板缓存和地图特征缓存在启动时一次性加载，运行时不重复分配大对象。日志线程的队列设置上限，超过上限时丢弃 DEBUG 级别事件，保留状态变化和错误。

13.5 安全停止流程

触发停止后，程序按固定顺序执行：设置停止标志，清空输入队列，释放所有按下的键和鼠标按钮，停止子流程，写入当前战斗的异常结束记录，写入会话结束时间和停止原因，关闭数据库，退出进程。

停止标志在输入线程、看门狗线程和主线程之间共享。任何线程检测到停止标志都不再提交新动作，已经在执行的动作立即完成释放步骤。

如果进程被强制结束，下一次启动时数据库会看到没有 ended_at 的会话记录，将其标记为异常结束。脚本不尝试修复上一场战斗的记录。

13.6 人工介入说明

进入 PAUSED 后，程序不再发送任何输入，控制台和日志显示暂停原因。使用者检查游戏画面、窗口状态和日志后，可以按配置的恢复热键继续，或直接关闭程序。

恢复热键默认 F11，与紧急停止 F12 分开，避免误触。恢复后状态机从 INIT 重新识别当前界面，不回到暂停前的状态。

14 测试与验收

14.1 单元测试

单元测试不启动游戏，使用构造的观察结果和假时钟驱动各模块。目标是把坐标、状态转移、调度边界这些容易出错且不需要实机验证的逻辑覆盖完整。

模块	测试内容
坐标变换	世界坐标与底图像素坐标往返一致；y 轴翻转正确；图像边界外坐标被拒绝；归一化区域换算在窗口缩放后保持一致
配置校验	缺失字段、类型错误、越界值、引用不存在的模板和出生点能给出准确的字段路径
地图数据	航线与出生点关联正确；重复编号被拒绝；航点超出边界被拒绝
航线选择	同一种子产生相同航线；权重为 0 的航线不会被选中；没有可用航线时返回明确错误
行为调度	所有采样结果落在钳位范围内；计时器到期顺序正确；攻击事件只选择开火或消耗品之一；消耗品达到上限后改为开火；停止标志生效后不再产生动作
船只校验	卡片匹配成功才选择；校验失败进入暂停；空槽判断正确
截图清理	战斗结束后删除对应目录；会话启动时清理遗留目录；数据库不写截图路径
死亡投票	各信号组合的判定结果符合规则；矛盾信号清零；战斗前 5 秒不判定
状态机	每条合法转移都能触发；非法转移进入异常恢复；超时事件在正确时间产生
输入队列	动作按顺序执行；取消后清空队列；复合动作不被其他动作打断
时间计算	单调时钟推进正确；长时间运行不出现累积误差

14.2 回放测试

回放测试使用录制的实机画面，覆盖港口、排队、加载、战斗、死亡、退出菜单、确认框和回港弹窗。每帧有手动标注的期望状态，回放时运行完整感知流程，比较识别结果和期望状态。

回放测试关注三类错误：状态识别错误、地图识别失败、舰船位置偏差。识别错误率超过 2% 时调整模板或搜索区域；地图识别失败时检查底图和特征缓存；位置偏差超过 5 个像素时检查小地图标定。

回放样本按客户端和分辨率保存，客户端更新后重新录制。样本不放入公开仓库，避免包含其他玩家信息。

14.3 标定精度验证

地图标定完成后，在底图上选取若干已知点，用标定工具预览世界坐标和像素坐标的换算，检查往返误差。误差主要来自 image_bounds 标定不准，要求往返误差小于 1 个像素。

地图识别的重投影误差中位数要求小于 2.5 个像素；出生点识别位置与配置出生点的距离要求小于 500 米，超过时检查标定和舰船标记识别。

导航点击精度通过实机验证：点击一个已知航点后，观察舰船是否朝目标方向移动。点击坐标与世界坐标的换算误差应小于 3 个像素，对应 12 公里地图上约 120 米，由 400 米的到达容差吸收。

方向验证：分别用正常和翻转预览检查出生点、航点是否落在游戏内对应位置。地图识别应能自动判断旋转和镜像；声明方向与识别结果不一致时以识别结果为准，并记录警告。

14.4 实机验收阶段

阶段	内容	通过标准
S1 环境	check_env 全部通过，截屏、输入和船只卡片匹配可用	窗口、截屏、DPI、点击、按键、船只卡片均正常
S2 单场	脚本确认船只和空装备，手动进入战斗，识别地图、出生点并导航，死亡后自动退出	单场闭环完成，无人工干预
S3 五场	自动开局、战斗、退出、回港、再开局连续五场	五场全部完成，无卡死超过 2 分钟
S4 多地图	配置三张以上地图和多个出生点，逐张验证识别和航线	每张地图识别成功，航线完成率不低于 90%
S5 连续运行	连续运行 4 小时或 20 场	完成率不低于 90%，无人工干预，异常均有记录

S2 是第一个可交付节点，用来验证主流程是否成立。S2 未通过时不进入 S3，避免在流程不稳定的情况下扩大测试范围。

14.5 验收指标

指标	目标值	测量方法
单场闭环成功率	S2 阶段 100%	人工观察并核对日志
连续场次完成率	20 场中至少 18 场完成	会话记录中 result 为 dead 或 finished 的场次比例
死亡到点击退出	中位数不超过 3 秒	battles 表时间差统计
死亡到回到港口	中位数不超过 45 秒	battles 表时间差统计
死亡误判	20 场中不超过 1 次	人工核对死亡截图和日志
地图识别成功率	已配置地图不低于 98%	每场识别结果与日志对照
出生点识别误差	不超过 500 米	识别位置与配置出生点距离
导航放弃率	不超过 5% 的场次	nav_abandon 事件统计
卡住事件	平均每场不超过 1 次	nav_stuck 事件统计
异常恢复成功率	不低于 80%	recovery 事件中恢复成功的比例
攻击事件开火比例	75%～95%	battle_actions 中 fire_burst 与 consumable 的数量比
截图清理	战斗结束后无残留目录	检查 screenshots 目录
人工干预次数	S5 阶段为 0	会话期间的人工操作记录

指标是验收依据，不是运行时的硬性限制。指标未达到时先分析日志，区分识别问题、配置问题和游戏行为变化，再决定修改设计还是调整参数。

14.6 测试记录

每次实机测试记录测试时间、客户端版本、地图、出生点、航线、开始和结束状态、异常截图路径和人工备注。测试记录保存在 `tests/records/`，不写入数据库，避免测试数据和正式运行日志混在一起。

客户端版本更新后，先跑回放测试，再跑 S2 单场，确认界面模板和退出步骤仍然有效，然后才继续长时间运行。

15 开发计划

15.1 阶段划分

开发按六个阶段推进，每个阶段有独立交付物和完成标准。阶段顺序按照“先工具、后主流程、再自动化”的原则安排，避免在界面行为尚未确认时写死逻辑。

阶段	内容	交付物	完成标准
M0 环境与工具	安装游戏，设置无边框窗口；实现截屏、配置加载、校验、check_env、标定工具、录制回放；标定船只卡片和空装备	可运行的检查与标定工具，一张地图和一条船只配置	check_env 通过，地图和船只标定完成，回放能识别录制画面
M1 单场闭环	实现状态机骨架、船只与装备校验、港口开局、死亡后退出、回港确认；地图和航线手动指定	可自动完成单场战斗的程序	手动开始战斗后，脚本能校验船只、导航、死亡退出并回到港口
M2 感知自动化	实现界面特征识别、地图识别、出生点识别、舰船位置识别、死亡投票	不依赖人工指定的完整感知链路	多场战斗中自动识别地图和出生点，死亡判定准确
M3 导航与行为	完善导航重试、卡住恢复、航线结束行为；实现攻击事件调度、开火与消耗品二选一、活动看门狗	稳定的导航和防挂机行为	S3 五场连续运行通过，导航放弃率和卡住次数达标
M4 多地图多客户端	扩充地图包，补齐 Steam 与 WG 客户端配置，完善多地图加载与选择	多地图数据和多客户端配置	S4 多地图验证通过，至少两个客户端可运行
M5 稳定性与验收	异常恢复、日志统计、资源限制、连续运行测试	完整可交付版本和测试报告	S5 连续运行通过，指标达到第 14.5 节要求

15.2 参考工作量

工作量按单人、课余时间投入估算，仅用于安排顺序，不作为承诺。M0 约 3～5 个工作日，M1 约 5～8 个工作日，M2 约 5～8 个工作日，M3 约 4～6 个工作日，M4 约 5～8 个工作日，M5 约 3～5 个工作日。

地图标定和实机验证的时间难以预估，每张地图的标定和航线调试可能需要半天到一天。M4 的工作量主要取决于地图数量和客户端数量，而不是代码复杂度。

如果课程或项目有固定截止时间，优先保证 M0 到 M3，单客户端、单地图或少量地图的完整闭环比多地图但流程不稳定更有价值。

15.3 依赖与顺序

M0 的工具是所有后续阶段的基础。没有截屏和标定，无法确认界面参数；没有录制回放，感知调整只能靠反复实机运行，效率很低。

M1 先用手动指定的地图和航线跑通流程，可以在感知还不完善时验证状态机和退出路径。M2 再把地图和出生点识别接进来，问题定位更容易。

M4 的多客户端适配放在主流程稳定之后。每个客户端都需要单独的截图、模板和步骤验证，过早开始会分散精力。

游戏客户端更新可能改变界面，任何阶段都应保留回放样本，更新后先跑回放，再跑单场，最后跑连续场次。

15.4 每阶段的风险检查

- M0 结束时确认：截屏不是黑屏，DPI 正确，标定坐标往返一致，输入在实机可用，船只卡片和空装备模板能稳定匹配。
- M1 结束时确认：死亡判定没有漏判，退出路径在多种死亡视角下都能走通，船只或装备校验失败时能暂停。
- M2 结束时确认：地图识别不会在多张地图之间混淆，出生点识别误差在容差内。
- M3 结束时确认：导航不会长时间点击无效位置，随机行为不会在非战斗界面触发，攻击事件始终只执行一种动作。
- M4 结束时确认：每个客户端配置独立可用，不依赖其他客户端的模板。
- M5 结束时确认：异常都有日志和状态记录，停止原因明确，人工介入步骤可执行。

16 待实机确认清单

以下项目依赖游戏实际画面和操作，必须在游戏安装完成后逐项验证。验证结果回填到本表，并同步修改相关配置。当前假设仅用于编码阶段的占位。

编号	待确认项	影响范围	验证方法	当前假设
V01	国服 360 客户端进程名与窗口标题	窗口查找	任务管理器查看进程，工具读取标题	WorldOfWarships.exe，标题含游戏名
V02	亚服 Steam 客户端进程名与窗口标题	客户端配置	同上	与国服相同或相近
V03	亚服 WG 客户端进程名与窗口标题	客户端配置	同上	与国服相同或相近
V04	港口联合作战入口、模式选择和开始按钮的位置与文案	开局流程	港口截图并逐项点击	按钮位于港口下方，模式可记忆
V05	快速开局是否可用，联合作战模式是否有明确指示	开局流程	连续两场观察模式是否保持，港口是否显示当前模式	游戏记住上一场模式，港口显示当前模式
V06	排队界面特征	状态识别	点击开始后截图	屏幕中部显示匹配进度

编号	待确认项	影响范围	验证方法	当前假设
V07	加载界面特征与地图名称位置	状态识别、地图识别	加载时截图	加载画面显示地图名称
V08	战斗 HUD 与小地图位置、大小	状态识别、导航	战斗截图并测量像素	小地图位于右下角，尺寸约 300 像素
V09	小地图是否显示全图、是否北向固定、是否可点击	导航	战斗中观察并点击测试	显示全图、北向固定、可点击设置航向
V10	自身舰船标记的形状、颜色和朝向表示	舰船定位	小地图放大截图	白色箭头，箭头指向航向
V11	血条位置和死亡后的变化	死亡判定	受伤和死亡时连续截图	死亡后己方血条消失
V12	死亡提示的文字和显示时长	死亡判定	死亡时录制画面	屏幕中央出现“你已被击沉”
V13	观战界面的特征	死亡判定	死亡后观察	出现观战目标信息面板
V14	ESC 菜单的退出入口和确认框	退出流程	死亡后手动操作截图	ESC→返回港口→确认
V15	死亡后能否立即退出，是否有等待时间	退出流程	死亡后立即按 ESC 测试	可以立即退出
V16	不同死亡视角下退出入口是否一致	退出流程	分别观察自己沉没、观战、全队失败	入口一致
V17	结算界面和返回港口按钮	战斗结束流程	存活到战斗结束并截图	结算后出现返回港口按钮
V18	回港弹窗种类与关闭方式	续场流程	回港后观察并截图	奖励、任务、首胜弹窗可点击关闭
V19	绝对坐标点击是否被游戏接受	导航、退出	check_env 点击测试	小地图点击可设置航向
V20	相对鼠标移动是否旋转视角及灵敏度	视角行为	check_env 移动测试	相对移动有效，灵敏度可调
V21	鼠标左键是否开火	开火行为	战斗中点击测试	左键开火
V22	消耗品键位、启用条件和冷却表现	消耗品行为	游戏设置查看按键，战斗中观察点击效果	栏位键位可配置，未满足条件时点击无效
V23	窗口失焦、最小化时的游戏表现	运行保障	切换窗口和最小化测试	失焦继续运行，最小化停止渲染
V24	客户端 UI 缩放对模板尺寸的影响	界面识别	调整 UI 缩放后截图对比	模板按参考分辨率采集，多尺度匹配可覆盖

编号	待确认项	影响范围	验证方法	当前假设
V25	断线和崩溃时的界面表现	异常处理	模拟断网或关闭进程	返回登录界面或进程退出
V26	游戏内 AFK 判定机制是否可观测	行为设计	长时间不操作并观察	无公开的客户端 AFK 提示
V27	外部底图与游戏小地图的特征匹配程度	地图识别	对每张地图运行识别测试	特征匹配内点数满足阈值
V28	wowsdb.com 地图库的图片分辨率、边框、图例和使用许可	地图素材	下载图片并与游戏小地图对比，核对站点条款	图片可下载，本地研究使用，标定时排除边框
V29	港口船只列表的打开方式、滚动方向和卡片样式	船只选择	港口截图并滚动测试	卡片可用模板匹配，列表可逐格滚动
V30	船只卡片在涂装、UI 缩放和不同港口状态下是否变化	船只识别	更换涂装和缩放后对比截图	模板稳定，必要时按涂装单独采集
V31	是否存在空信号旗、空经济加成卡的预设，以及选择方式	装备检查	港口装备界面创建并选择预设	可以创建空预设，脚本可选中
V32	空信号旗槽和空经济加成槽的外观特征	装备检查	逐个槽位截图	空槽模板可稳定命中
V33	装备清除按钮是否存在，点击后果是什么	装备检查	手动测试清除操作	默认不自动清除，只做校验
V34	每场开始前游戏是否会改变当前船只或装备	每场校验	连续多场观察船只和槽位	游戏不会自动改变
V35	游戏内小地图与外部底图的方向关系	地图识别、导航	进入不同地图，对照上下、左右和旋转后的截图	可能存在上下颠倒、镜像或 90 度旋转
V36	小地图是否随镜头旋转，能否固定北向	坐标变换	转动镜头观察小地图	可以固定北向
V37	地图识别在翻转和镜像底图上的稳定性	地图识别	对每张地图分别用正常、翻转、镜像图片测试	ORB 加镜像模板可以稳定识别
V38	翻转后航线与出生点在游戏内是否对应	导航	在翻转地图上跑一条航线，对照游戏内位置	世界坐标不受方向影响，预览一致

V27 是外部底图方案能否成立的关键。如果某张地图的底图与游戏小地图特征差异过大，需要改用游戏内截图生成模板，或者放弃该地图。这项验证应在 M0 阶段完成，不要等到 M4 才发现地图不可用。

17 风险与运行边界

17.1 账号风险

初步设计文档第 7 章已经说明，使用自动化脚本接管游戏操作违反游戏官方的第三方软件政策，处罚对象是账号，国服和亚服的处罚力度不同。脚本的行为随机化只能降低客户端层面的挂机判定风险，不能保证规避服务器端对自动化行为的识别。

本项目仅用于本地技术研究。使用可弃小号运行，不在主力账号上运行；限制每日场次和单次运行时长；不公开传播、不出售脚本；同机避免多账号频繁切换。这些限制写入会话配置和运行说明，而不是只作为建议。

第 7 章的处罚规则和风险描述来自公开资料，可能随游戏版本和运营策略变化。开始运行前应自行核对当前官方政策。

17.2 技术风险

风险	影响	应对
外部底图与小地图特征不匹配	地图识别失败，导航无法启动	M0 阶段逐图验证；失败时改用游戏内截图模板或放弃该地图
客户端界面更新	模板失效，状态识别和退出失败	保留回放样本，更新后先回归；模板和步骤全部配置化
多客户端差异大	配置和模板工作量成倍增加	先完成国服，再逐个客户端标定；不承诺所有客户端同时可用
绝对点击被游戏忽略	导航和退出按钮点不中	保留 SetCursorPos 加鼠标事件的后备实现
自身标记误识别	导航位置错误、卡住判定失效	时序跟踪、颜色约束、失败降级和放弃导航
船只卡片或装备槽模板失效	选错船只、装备校验失败、脚本暂停	每场轻量校验；模板失效时暂停，不猜测；客户端更新后重新标定
死亡误判或漏判	提前退出或卡在战斗	多信号投票、截图记录、阈值可配置
小地图旋转或缩放	坐标变换失效	要求游戏设置固定北向和固定缩放；不支持时暂停
服务端识别自动化行为	账号处罚	接受风险，使用小号，限制运行时长和场次
Python 与 OpenCV 版本兼容问题	截屏或图像处理异常	M0 阶段做兼容性检查，优先调整依赖版本
长时间运行内存增长	进程变慢或崩溃	缓存预分配、队列上限、会话时长上限

风险	影响	应对
底图方向与游戏内小地图不一致	地图识别失败，航线镜像或颠倒	标定工具提供方向选择和翻转预览；识别时自动检测旋转和镜像；不一致时以识别结果为准并暂停

17.3 运行前提

脚本运行需要满足以下前提，任一不满足时结果不可靠：游戏以 2560×1600 无边框窗口运行；UI 缩放和游戏内小地图设置固定；游戏窗口保持前台；操作系统不锁定、不休眠；网络稳定；客户端版本与标定时的版本一致；账号已拥有配置船只，信号旗和经济加成卡已清空，或者已创建空装备预设。

这些前提不满足时，程序可能进入暂停或异常恢复，而不是自动修正。运行前检查环境是使用者的责任。

17.4 范围与责任

本文档描述的是本地单机自动化技术方案，不涉及绕过反作弊、修改客户端、读取游戏内存或干扰其他玩家。脚本不用于商业用途，不对外提供服务。

使用脚本产生的账号处罚、数据损失和其他后果由使用者承担。实现和运行过程中发现新的行为特征或风险，应补充到本文档第 16 章和第 17 章，而不是只记录在代码注释里。

附录 A 核心接口签名

以下签名用于统一实现阶段的模块边界。参数和返回值可以按实际编码调整，但模块职责和依赖方向不改变。

```
# 感知
class PerceptionEngine:
    def capture(self, region: NormalizedRect | None = None) -> np.ndarray
    def match_feature(self, feature_id: str) -> MatchResult
    def detect_state(self) -> str | None
    def identify_map(self, minimap: np.ndarray) -> MapMatch | None
    def locate_own_ship(self, minimap: np.ndarray) -> ShipPosition | None
    def check_death(self, frame: np.ndarray) -> DeathSignals
    def observe(self) -> Observation

# 状态机
class StateMachine:
    def step(self, observation: Observation, now: float) -> list[Action]
    def force(self, state: str, reason: str) -> None
    def can_transition(self, src: str, dst: str) -> bool

# 导航
```

```
class Navigator:
    def start(self, route: Route, calibration: BattleCalibration) -> None
    def tick(self, observation: Observation, now: float) -> list[Action]
    def stop(self) -> None

# 行为
class BehaviorScheduler:
    def start(self, profile: RandomProfile, now: float) -> None
    def tick(self, now: float) -> list[Action]
    def stop(self) -> None

# 输入
class InputExecutor:
    def submit(self, actions: list[Action]) -> None
    def release_all(self) -> None
    def stop(self) -> None

# 存储
class ConfigLoader:
    def load_global(self, path: str) -> GlobalConfig
    def load_profile(self, path: str) -> ClientProfile
    def load_maps(self, directory: str) -> dict[str, MapPack]
    def validate(self) -> list[ValidationIssue]

class Database:
    def begin_session(self, profile_id: str, config_hash: str) -> int
    def begin_battle(self, session_id: int, map_id: str | None) -> int
    def log_action(self, battle_id: int | None, action_type: str, detail: dict) -> None
    def log_state(self, session_id: int, battle_id: int | None, state: str, confidence:
float, detail: dict) -> None
    def end_battle(self, battle_id: int, result: str, exit_error: str | None) -> None
    def end_session(self, session_id: int, stop_reason: str) -> None
```

附录 B 关键流程伪代码

B.1 主循环

```
while not stop_flag:
    now = monotonic()

    if emergency_stop_pressed() or heartbeat_expired(now):
        request_stop("safety")
        continue

    observation = perception.observe()

    if observation.window_missing:
        state_machine.force("PAUSED", "window_missing")
        continue

    actions = state_machine.step(observation, now)
    if actions:
        executor.submit(actions)

    log_observation_if_debug(observation)
    sleep_until_next_tick()
```

B.2 死亡退出

```
death = perception.check_death(frame)
if death.score >= 0.75 and death.stable_ms >= 800:
    db.log_action(battle_id, "death_detected", death.to_dict())
    save_screenshot("death")
    navigator.stop()
    behavior.stop()
    state = "DEAD_PENDING_EXIT"
    wait(random.uniform(0.8, 2.0))

run_steps(profile.exit_sequence)
if port_visible():
    db.mark_returned_to_port(battle_id)
    state = "POST_EXIT_INTERFACE"
```

B.3 导航确认

```
  if pending_click is None:
      target = route.waypoints[index]
      executor.submit([click_screen(world_to_screen(target))])
      pending_click = target
      click_position = ship.position
      click_time = now
  elif now - click_time >= confirm_window:
      moved = distance(ship.position, click_position) >= move_confirm_m
      if moved:
          pending_click = None
          retries = 0
      else:
          retries += 1
          pending_click = None
          if retries > max_retries:
              skip_waypoint()
              retries = 0
```